

PACKET CAPTURE: OVERVIEW AND BASICS

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심
MMcN

멀티미디어 융합 연구실

Mobile Multimedia convergene Network Lab.
<http://mmcn.ajou.ac.kr>



Contents

- ❖ Sockets
- ❖ Packet Capture Basics
- ❖ Understanding Libpcap and Npcap
- ❖ Pcap Data Structure
- ❖ Example – Captured File Analysis

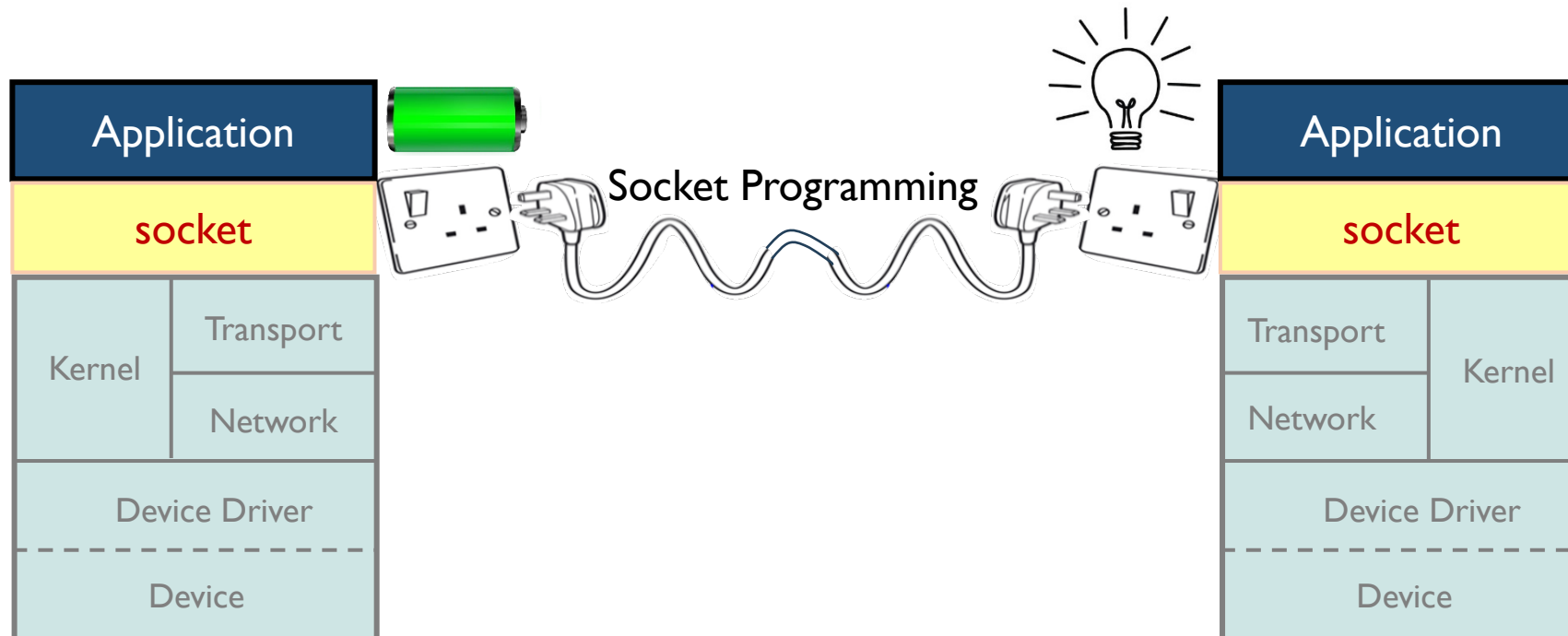


SOCKETS

Network Application Programming

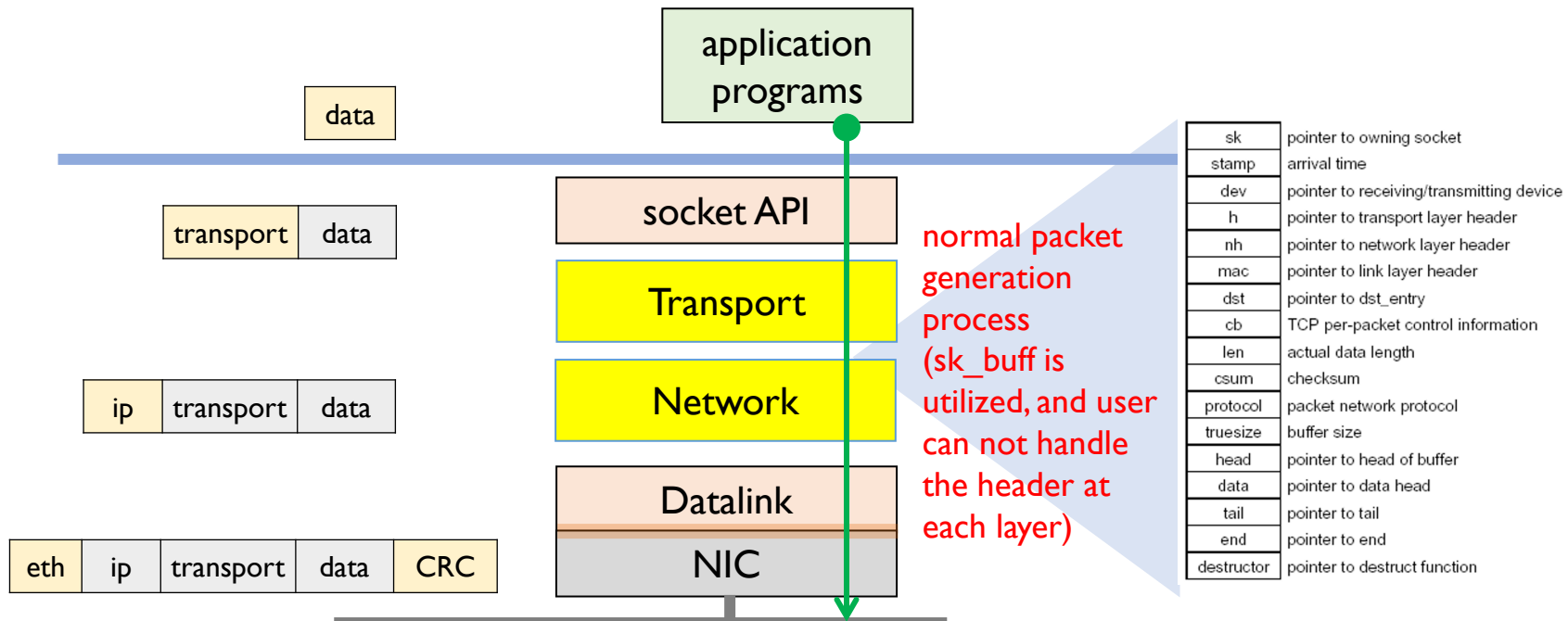
❖ Sockets provide

- network programming interface between
 - processes at application layers on the network
 - through TCP/IP network kernel (OS), but don't care of it



Standard Socket (1/3)

- ❖ In standard sockets,
 - the SDU from upper layer is encapsulated at a layer
 - TCP/UDP, IP, and Frame headers are automatically added, and
 - user don't need to aware the existence of TCP/IP layer

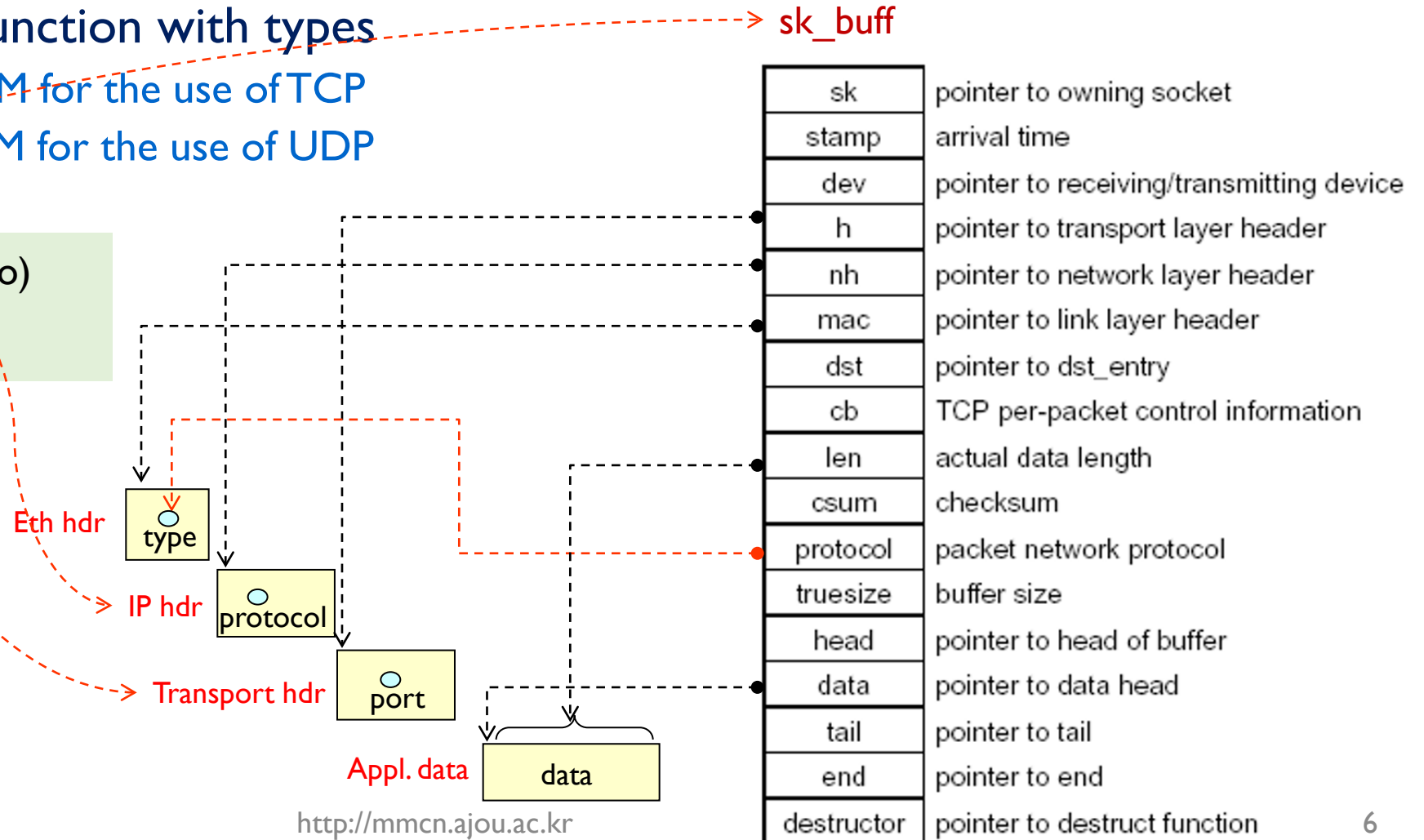


Standard Socket (2/3)

❖ To create standard sockets

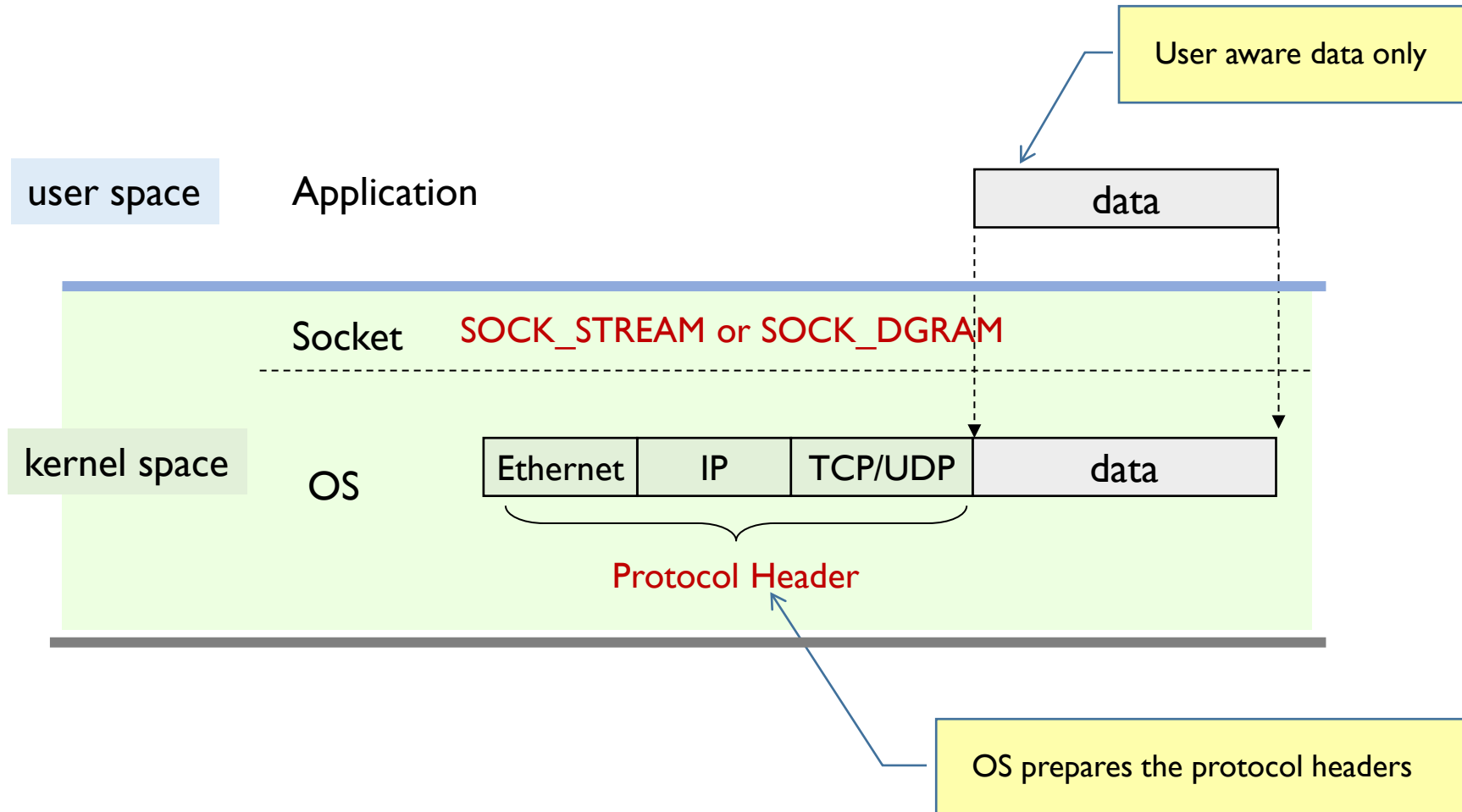
- use `socket( )` function with types
 - `SOCK_STREAM` for the use of TCP
 - `SOCK_DGRAM` for the use of UDP

```
sd = socket (dm, type, proto)
z = bind (sd, addr, len)
```



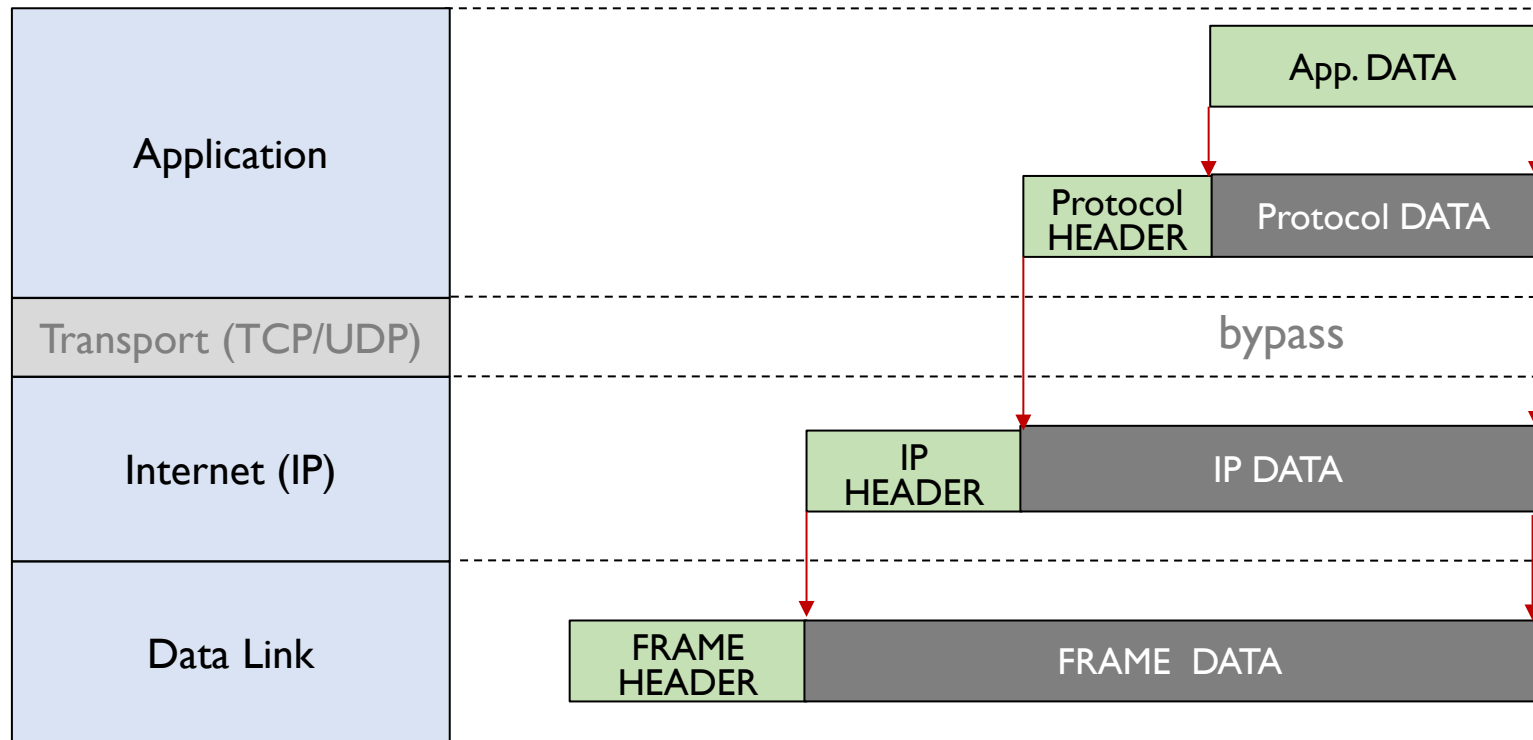
Standard Socket (3/3)

❖ SOCK_STREAM and SOCK_DGRAM



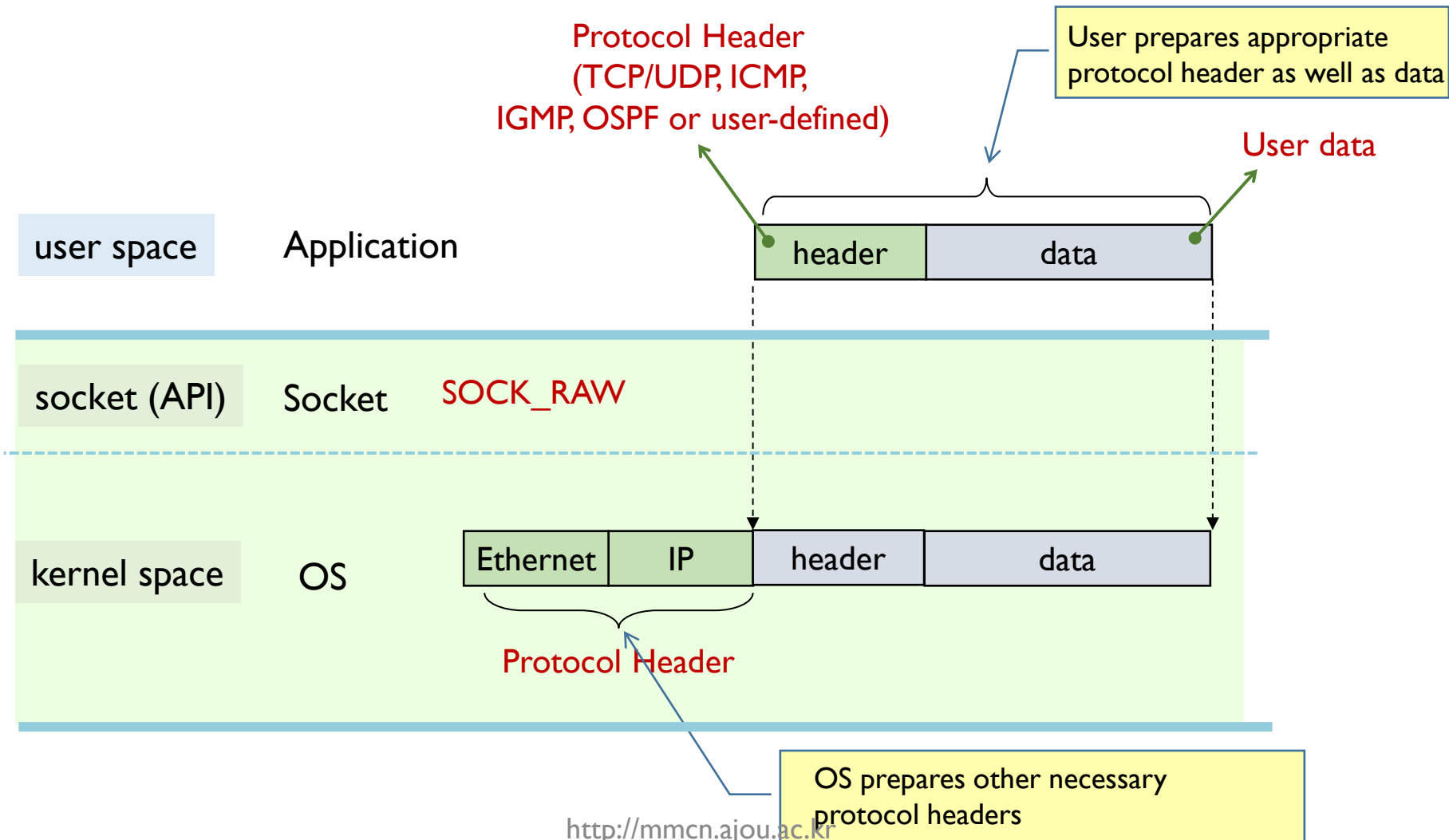
Raw Socket (1/2)

- ❖ A **raw socket** is
 - a type of socket that allows **bypass the transport layer**
 - user should prepare IP data part with appropriate headers and data



Raw Socket (2/2)

❖ RAW Socket

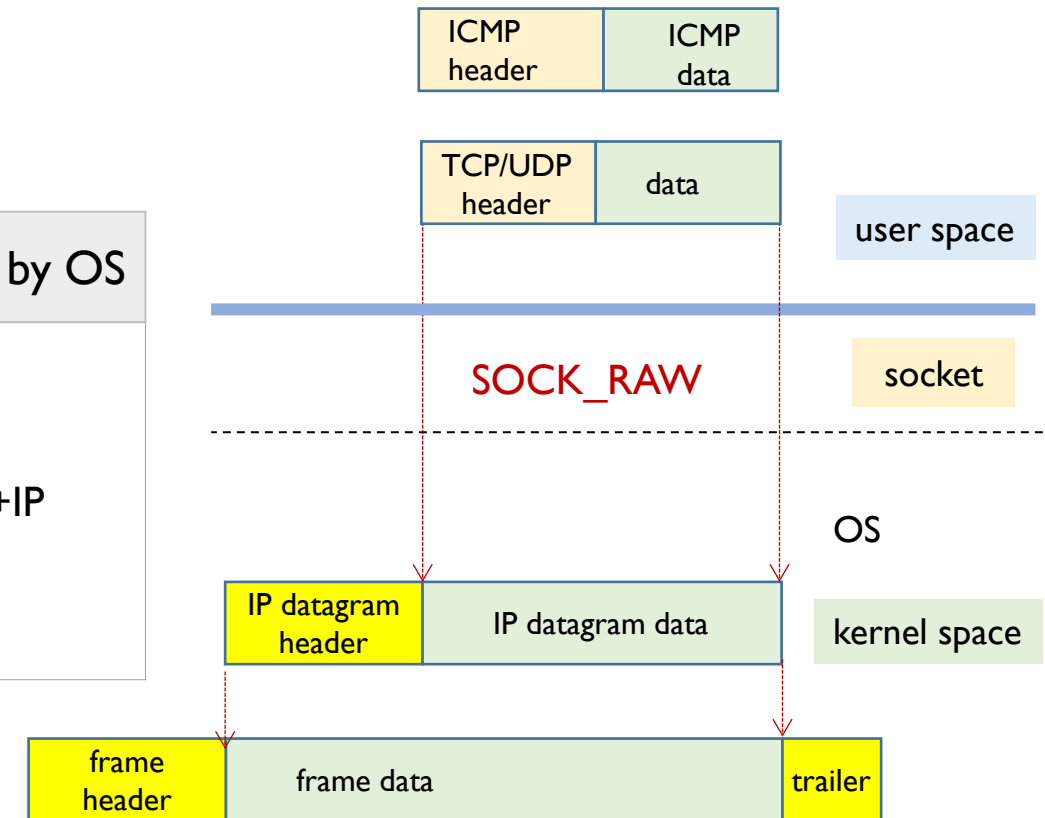


RAW Socket with IP_HDRINCL option (1/3)

❖ SOCK_RAW **without** IP_HDRINCL option

- it bypasses Transport layer only
- OS prepares IP header at network layer

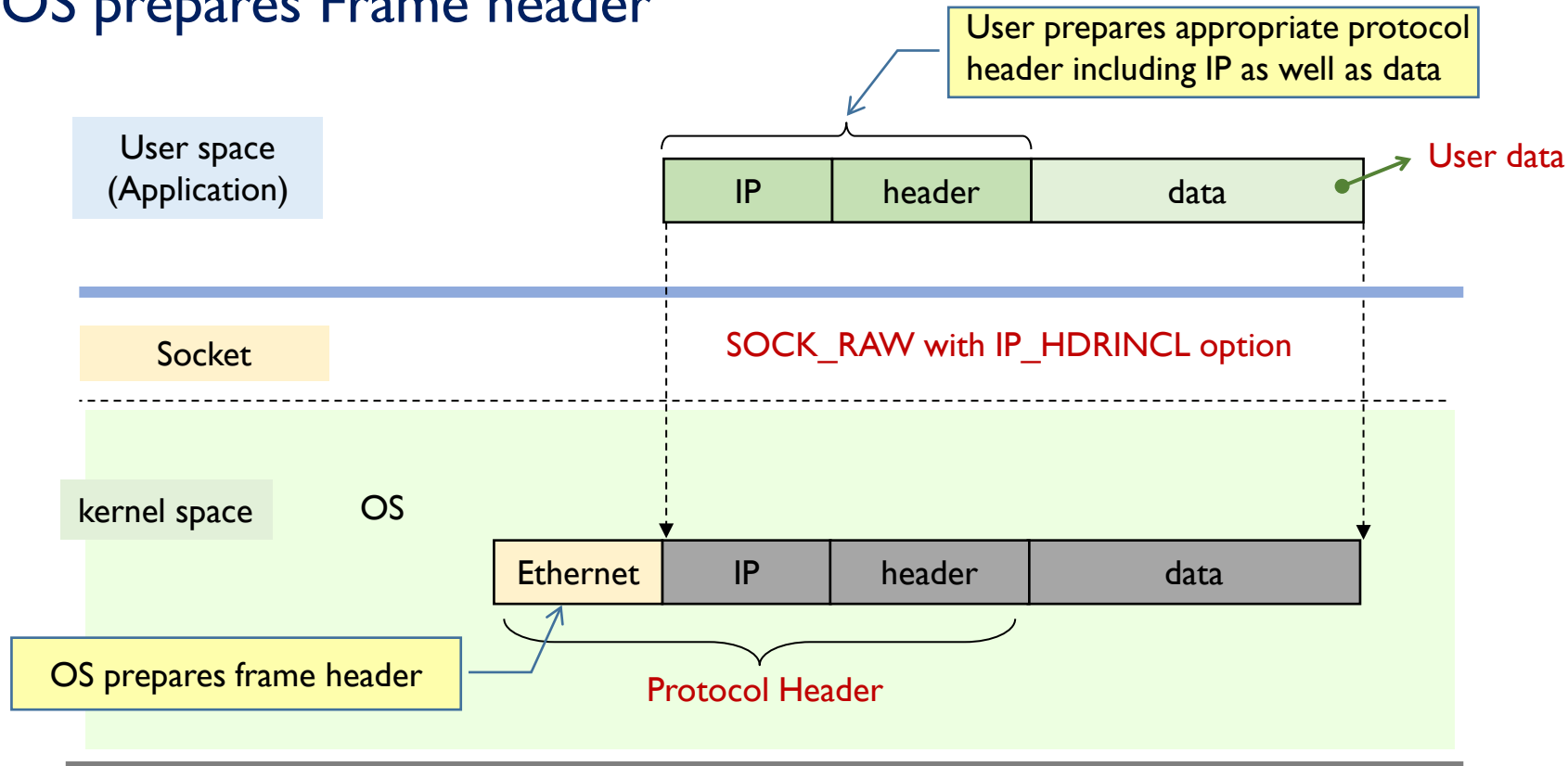
Protocol	Header given by Application	Header by OS
IPPROTO_ICMP	ICMP (ICMP Header + ICMP Data)	Eth+IP
IPPROTO_IGMP	IGMP (IGMP Header + IGMP Data)	
IPPROTO_TCP	TCP (TCP Header + App. Data)	
IPPROTO_UDP	UDP (UDP Header + App. Data)	
user defined	user defined header (IP + USER)	



RAW Socket with IP_HDRINCL option (2/3)

❖ SOCK_RAW with IP_HDRINCL option

- it bypasses both transport and IP layer
- OS prepares Frame header





RAW Socket with IP_HDRINCL option (3/3)

❖ Limitations of IP_HDRINCL option

- IP_HDRINCL only allows for creating IP headers on outgoing packets
 - it does not provide any functionality for capturing or analyzing incoming traffic
 - so, it can't be used for packet analysis
- IP_HDRINCL requires IP header by the application for each packet sent
 - it does not inherently handle lower layers, such as the Ethernet layer
- IP_HDRINCL depends on system-specific behavior
 - Some OSs may not fully support or implement this option in a consistent way
 - Windows does not support TCP packet transmission with IP_HDRINCL option

❖ libpcap

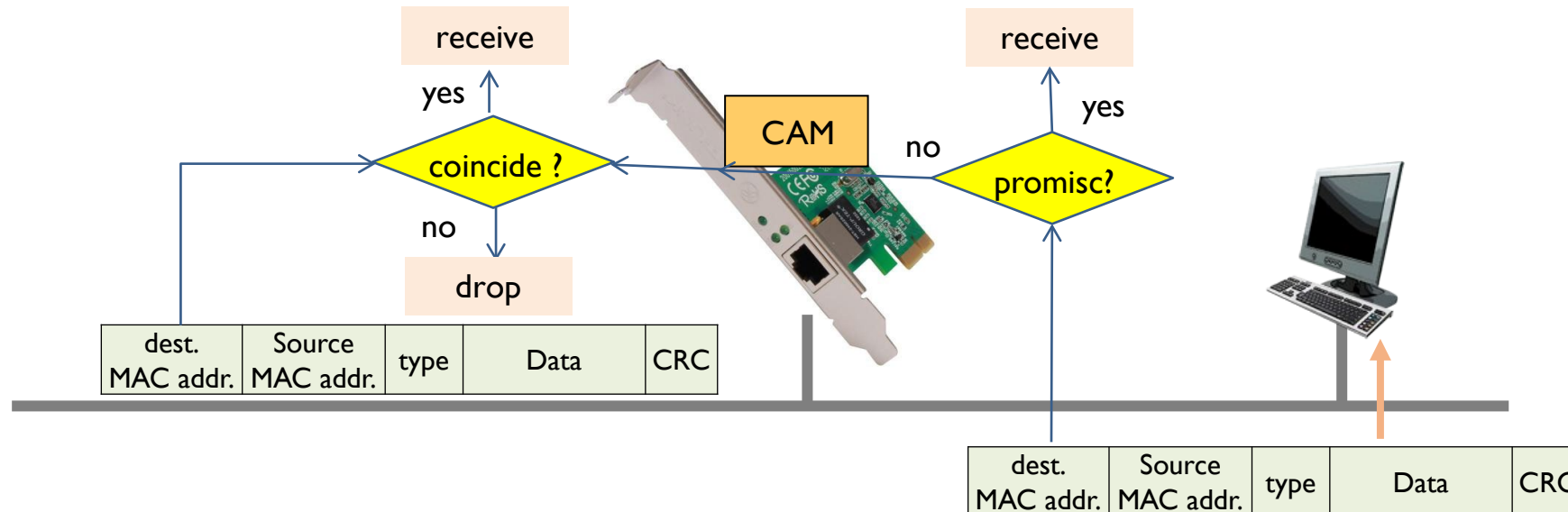
- preferable and comprehensive packet capture and analysis library
 - compared to IP_HDRINCL, which is more specialized and limited to sending custom IP packets only



PACKET CAPTURE BASICS

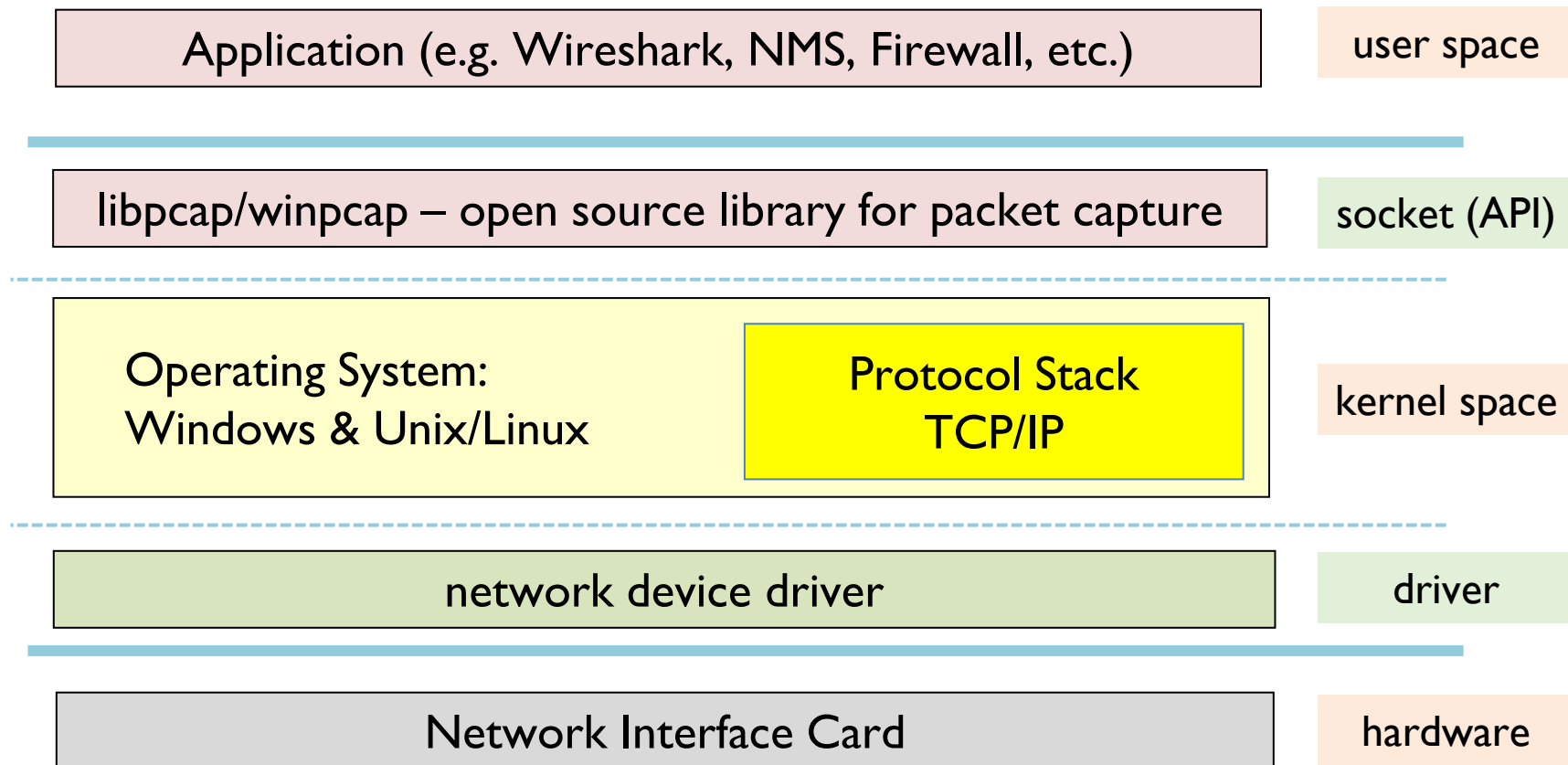
Network Interface Card (NIC)

- ❖ NIC accepts frames with their destination addresses
 - its MAC address – stored in EEPROM
 - multicast addresses – stored in Content Addressable Memory(CAM), and
 - broadcast address
- ❖ Promiscuous Mode
 - NIC accepts all frames it receives



Packet Capture Structure

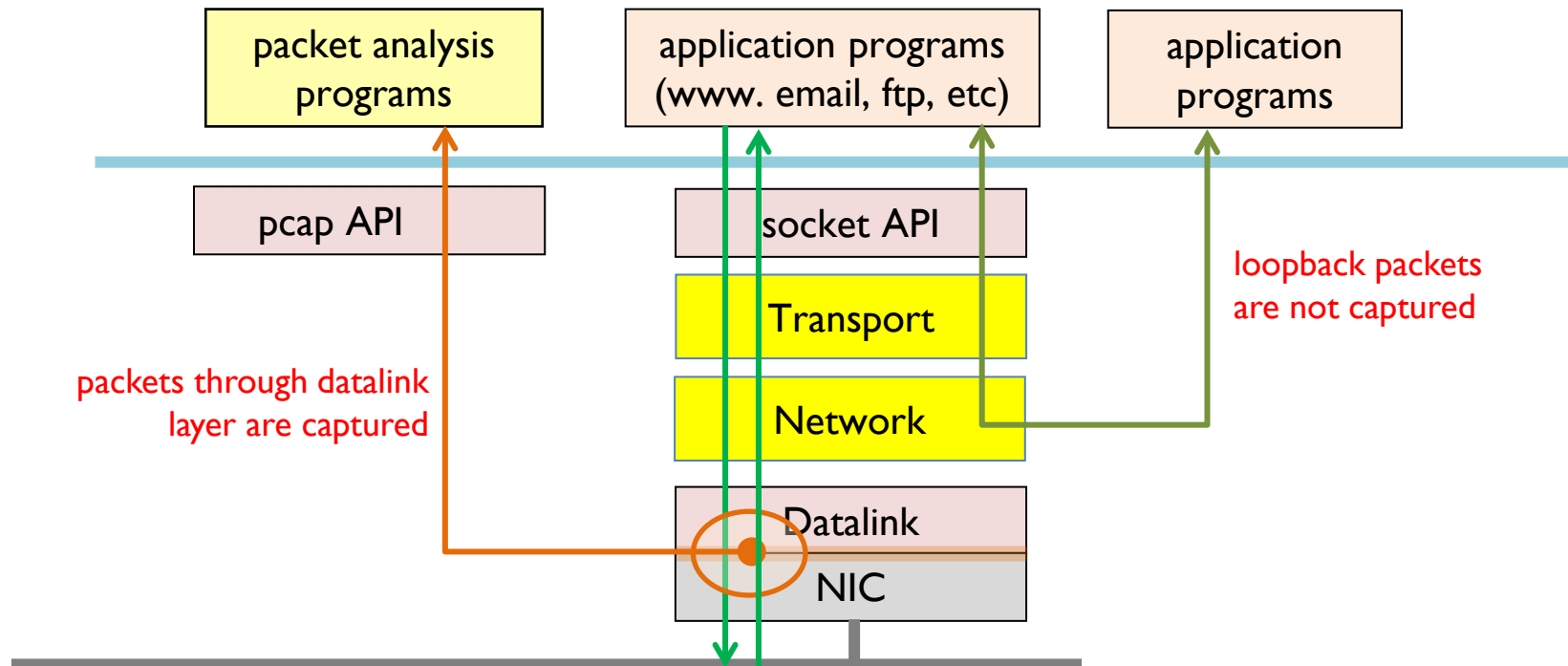
❖ Wireshark, OS, and Pcap library



Packet Capture Structure

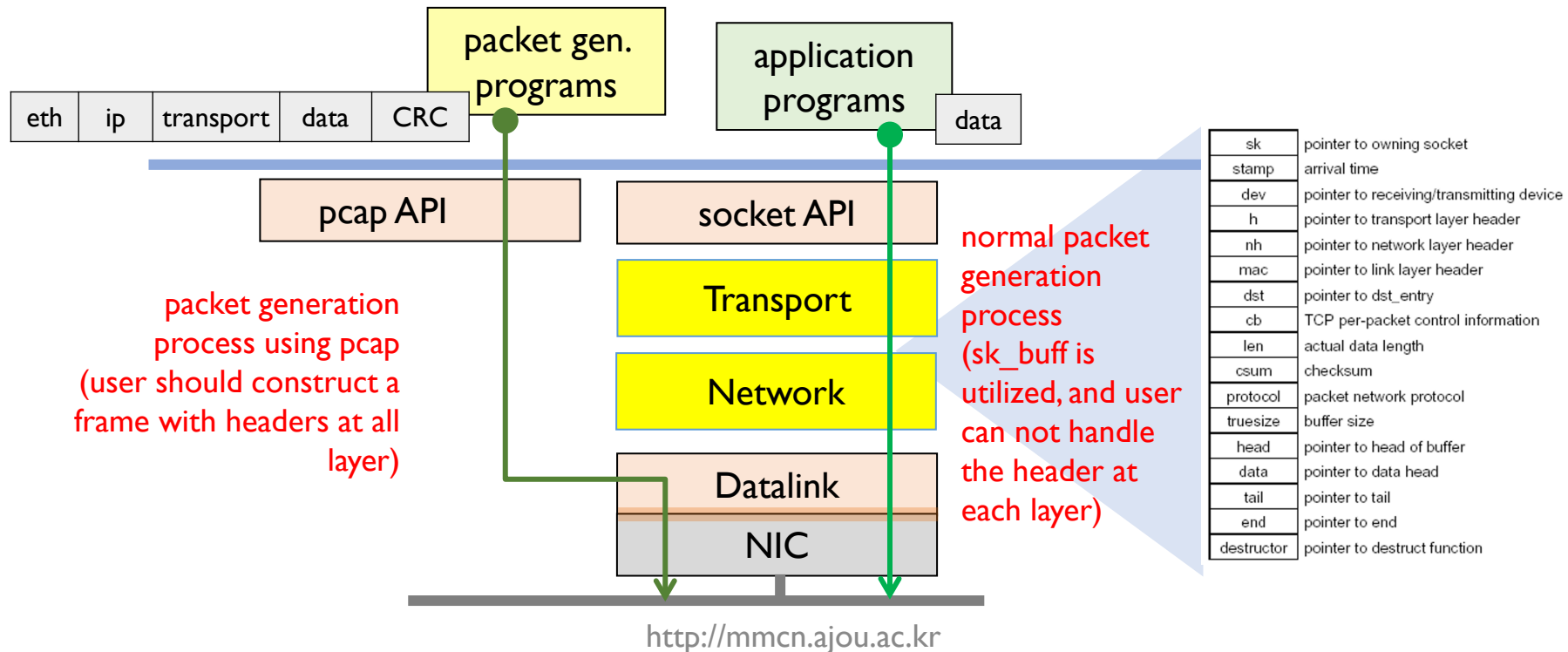
❖ Packet Capture Layer

- Packets are **not captured at all protocol layers**,
- Packets **in and out through NIC (data link layer)** are captured.
 - with pcap library, frames received and transmitted at datalink layer are delivered to application programs by passing tcp/ip stack



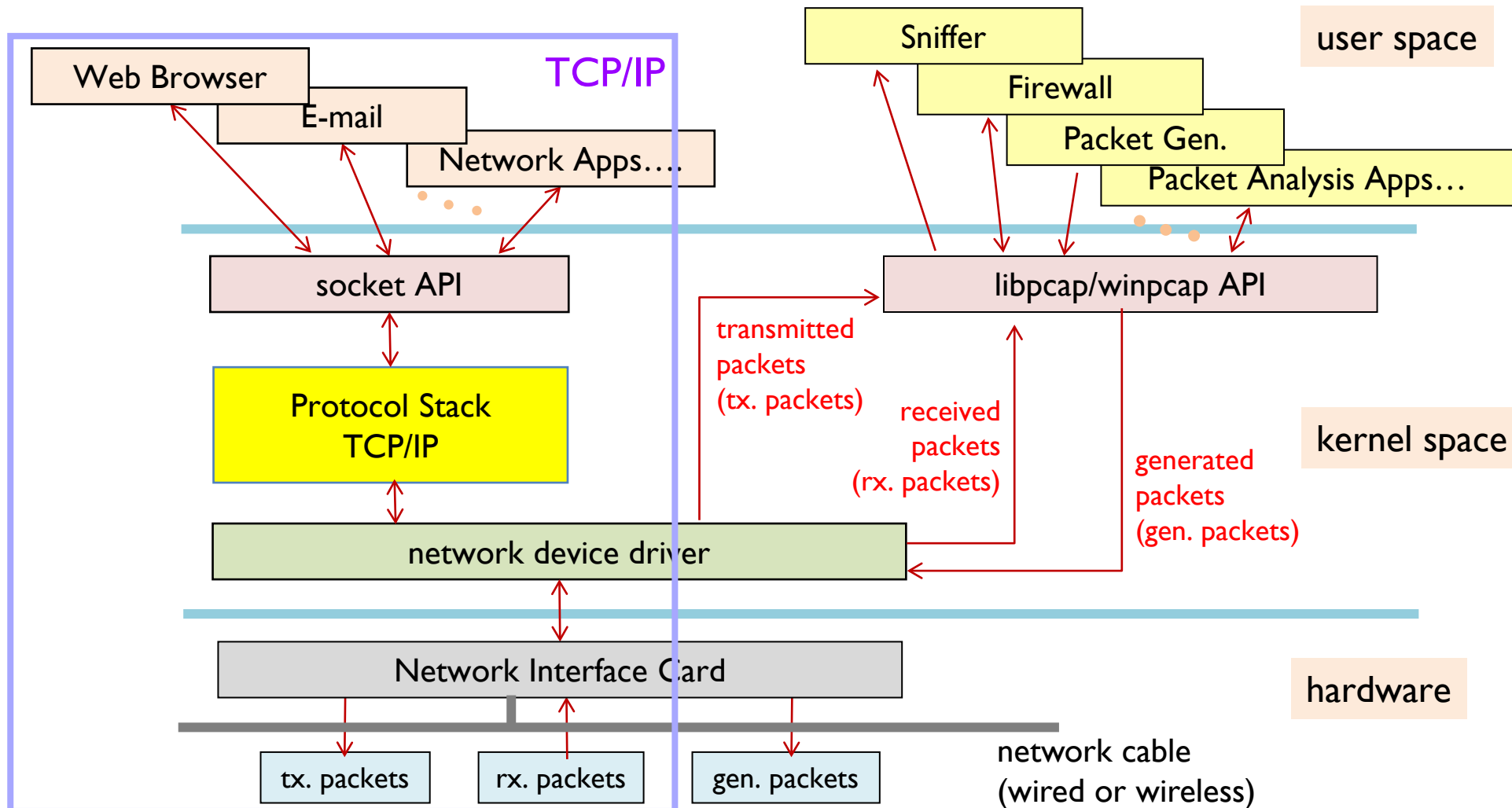
Packet Generation Structure

- ❖ pcap API provides packet generation process
 - an application program
 - constructs a full frame format with headers at all layers including application data, then
 - sends it through NIC directly using pcap library



Packet Analysis Process

❖ Packet sending, capture, and generation



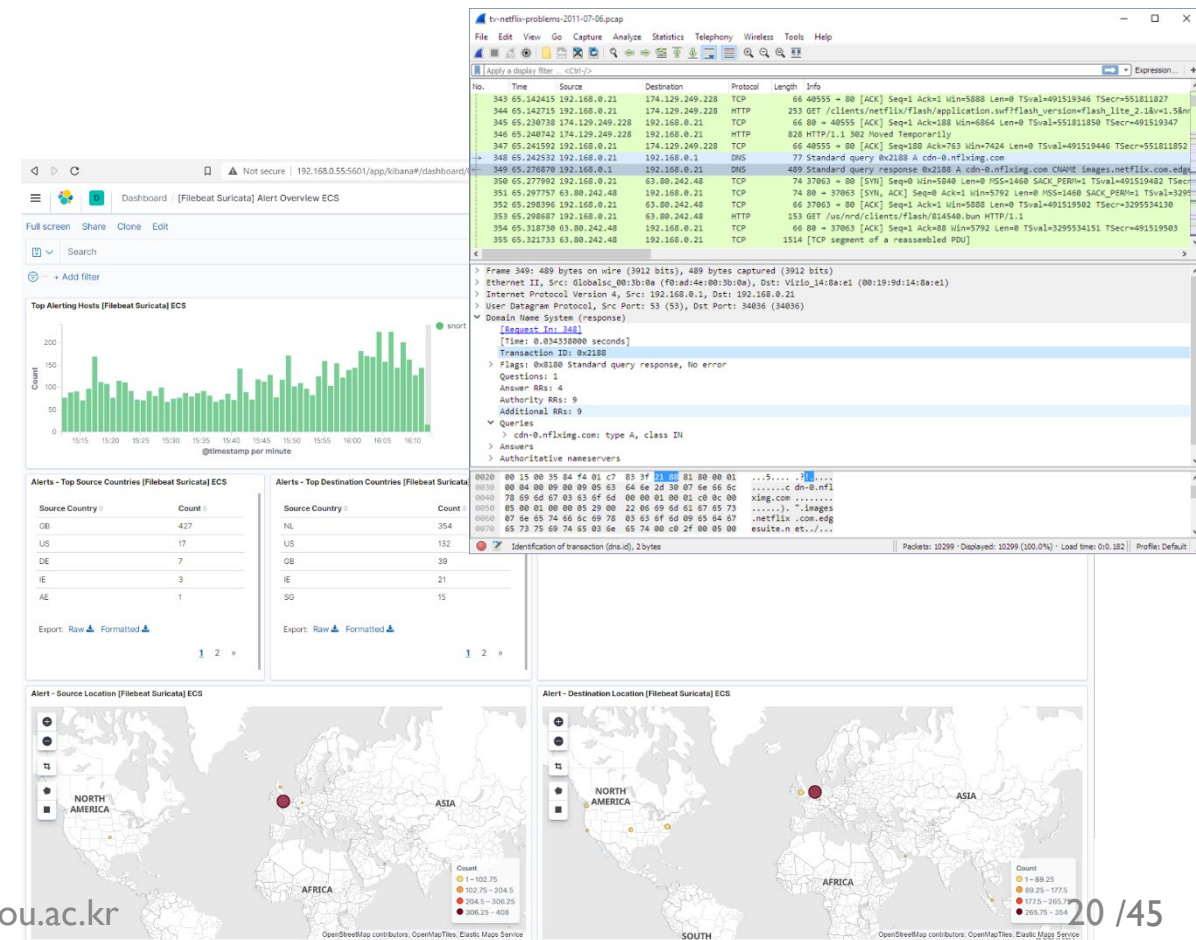
UNDERSTANDING LIBPCAP AND NPCAP



pcap library

❖ pcap (packet capture)

- open source library providing application programming interface (API) for packet-capture and network analysis
- written in C/C++ basically
- capable of
 - to capture network packets
 - to save captured packets
 - to analyze the real-time and saved packets
 - to inject packets, and so on



libpcap on Linux (1/2)

❖ libpcap on Linux

- developed in 1994
 - by the tcpdump developers in the Network Research Group at Lawrence Berkeley Laboratory
 - <http://www.tcpdump.org>
 - <https://github.com/the-tcpdump-group>
- installed by default on the BSDs, macOS, and Ubuntu Linux
 - Latest Releases
 - tcpdump: version: 4.99.5, released on Aug. 2024
 - libpcap: version 1.10.5, released on Aug. 2024
 - Checking the installation of libpcap



```
tcpdump --version
```

- ex)

```
bhroh@mmcn:~$ tcpdump --version
tcpdump version 4.99.4
libpcap version 1.10.4 (with TPACKET_V3)
OpenSSL 3.0.13 30 Jan 2024
```



libpcap on Linux (2/2)

- ❖ libpcap development package: libpcap-dev
 - includes the header files and development tools
 - needed to use the libpcap library in C or C++ programs
 - allows programs to access libpcap functionality
 - by providing files such as pcap.h
 - note: libpcap itself is a packet capture library used for capturing and analyzing network traffic. If the libpcap-dev package is not installed, pcap.h will be missing, leading to compilation errors when libpcap functions are used.
 - install the libpcap-dev

```
sudo apt update
sudo apt install libpcap-dev
```



Npcap on Windows

❖ WinPcap

- the Windows version of libpcap
 - <https://www.winpcap.org/>
- still available, but has not been maintained or updated since 2013

❖ Npcap

- an architecture for packet capture and network analysis for Windows OS
 - all of WinPcap's packet capture and injection features are included,
 - with a few great additions like raw 802.11 frame capture
- Npcap facilities
 - capture raw packets
 - filter the packets according to user-specified rules
 - transmit raw packets to the network
 - gather statistical information on the network traffic engine
 - support for remote packet capture.

Npcap on Windows

❖ Comparisons of features for Npcap and WinPcap

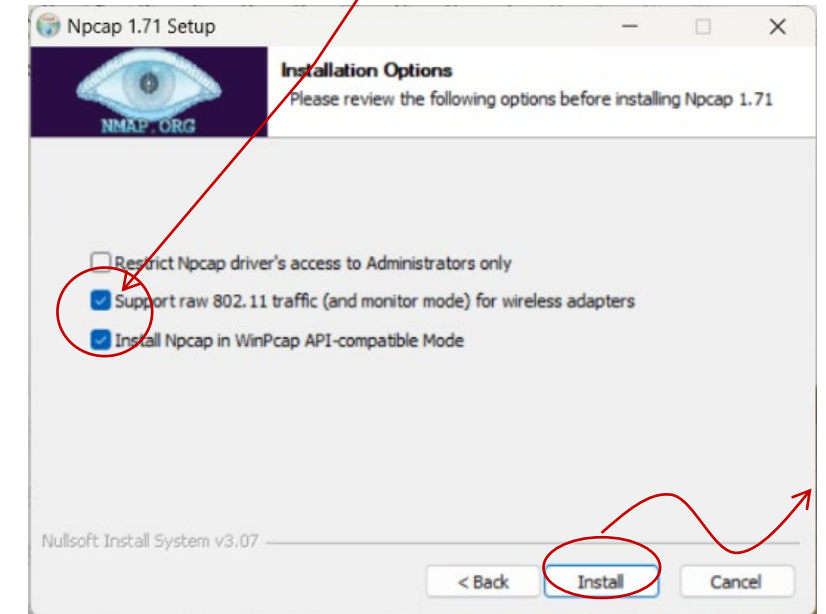
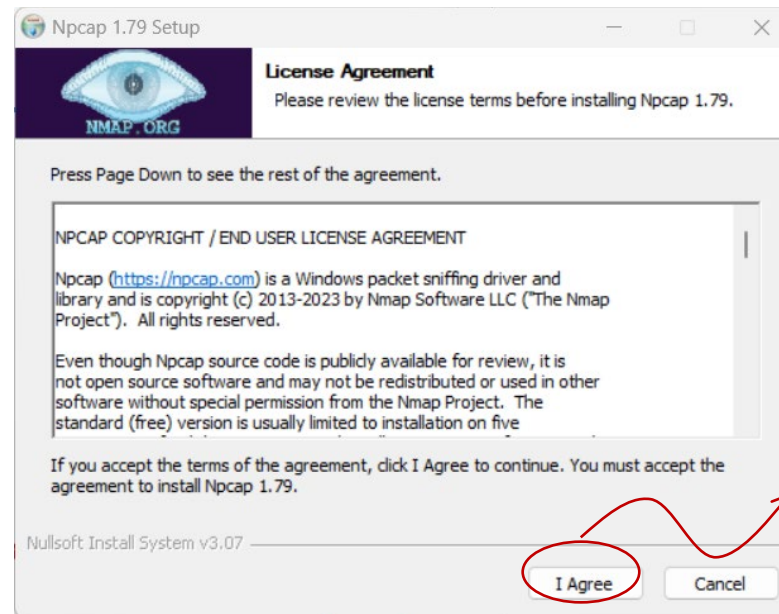
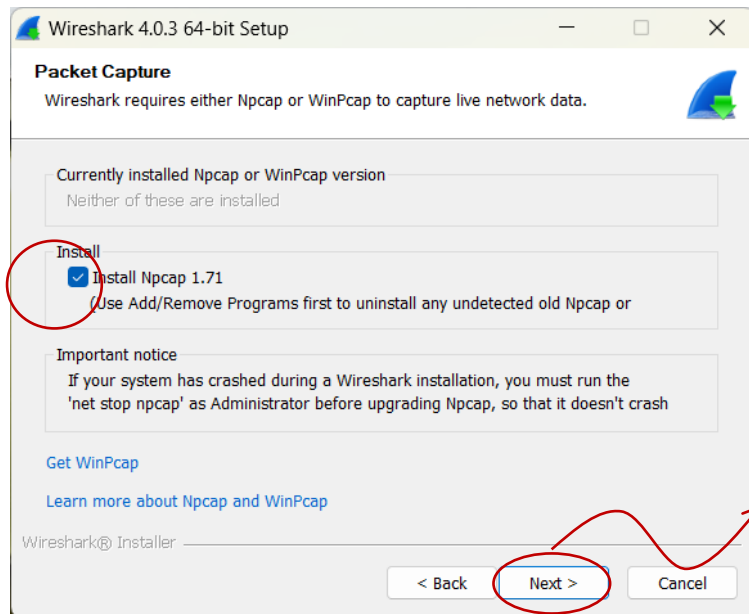
	Feature	Npcap	WinPcap
information	Actively maintained and supported	Yes	No (terminated)
	Latest release	Sep. 16, 2024	Mar. 8, 2013
	libpcap version	1.10.4 (2023)	1.0.0 (2008)
	License	Free for personal use	BSD-style
	Supported commercial/redistributable version	Yes (Npcap OEM)	No (terminated)
security	EV SHA-256 code signing	Yes	No
	Limit access to administrators (optional)	Yes	No
Basic Features	Packet capture with the cross-platform libpcap API	Yes	Yes
	Link-layer packet injection	Yes	Yes
	Source code available	Yes	Yes
Advanced Features	Capture raw 802.11 frames	Yes (with many adapters)	Yes (with specialized AirPcap)
	Capture Loopback traffic	Yes	No
	Inject Loopback traffic	Yes	No

Npcap on Windows

❖ install Npcap

- install during Wireshark installation
- download Npcap installer at <https://npcap.com/#download>
 - Npcap 1.80 installer for Windows 7, 8, 8.1, 10, 11 (x86, x64 and ARM64)
 - Latest development source: Github (<https://github.com/nmap/npcap>)

only for 802.11 WiFi packet capture, but the wifi adaptor should support monitor mode. Find adaptors in monitor mode at https://secwiki.org/w/Npcap/WiFi_adapters



during wireshark install

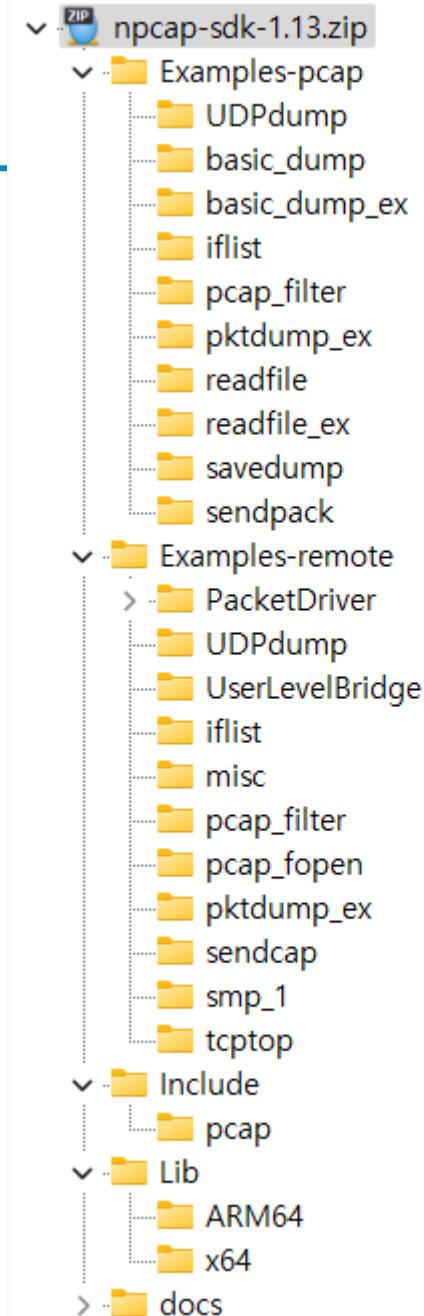
Npcap Programming

❖ Npcap SDK

- download Npcap SDK
 - at <https://npcap.com/#download>
 - Npcap SDK 1.13 (ZIP)
- unpack the SDK

❖ Resources

- Development Tutorial
 - <https://nmap.org/npcap/guide/npcap-tutorial.html>
- User's Guide
 - <https://nmap.org/npcap/guide/>
- WiFi adaptors in monitor mode
 - https://secwiki.org/w/Npcap/WiFi_adapters
- Comparisons to WinPcap
 - <https://nmap.org/npcap/vs-winpcap.html>



Npcap local capture application example codes

Npcap remote capture application example codes

Libraries and Include files for Npcap programming

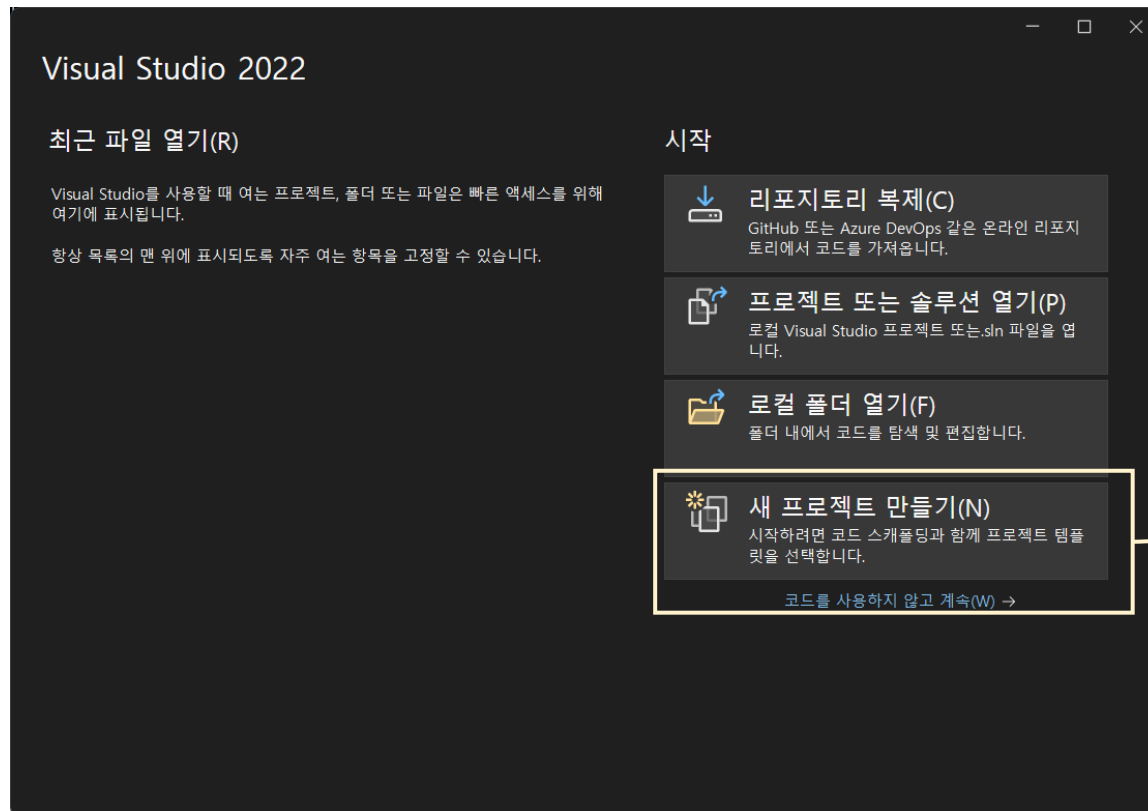


Npcap Programming with Visual Studio

- ❖ Configuring MS Visual Studio compilation environments
 - Include the file <pcap.h> at the beginning of every source file
 - that uses the functions exported by library.
 - Add directories for include and library files, for example
 - NpcapSDK\include for include directory
 - NpcapSDK \lib for library directory
 - Include the following preprocessor definitions
 - WPCAP for Win32 specific functions of Npcap
 - HAVE_REMOTE for the remote capture capabilities of Npcap
 - Set the options of the linker to include the wpcap.lib
 - Set the options of the linker to include the winsock library file ws2_32.lib.

Npcap Programming with Visual Studio

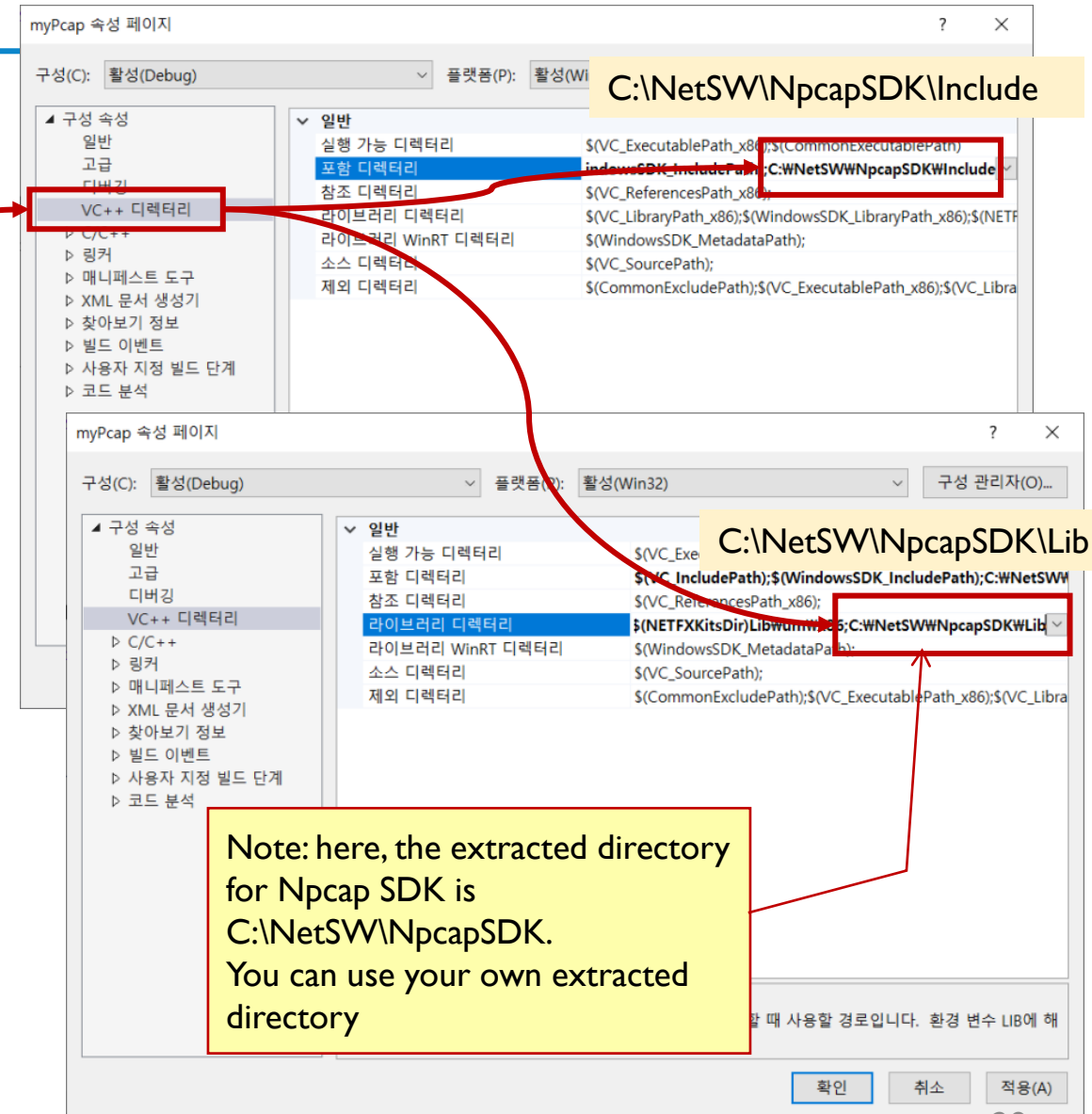
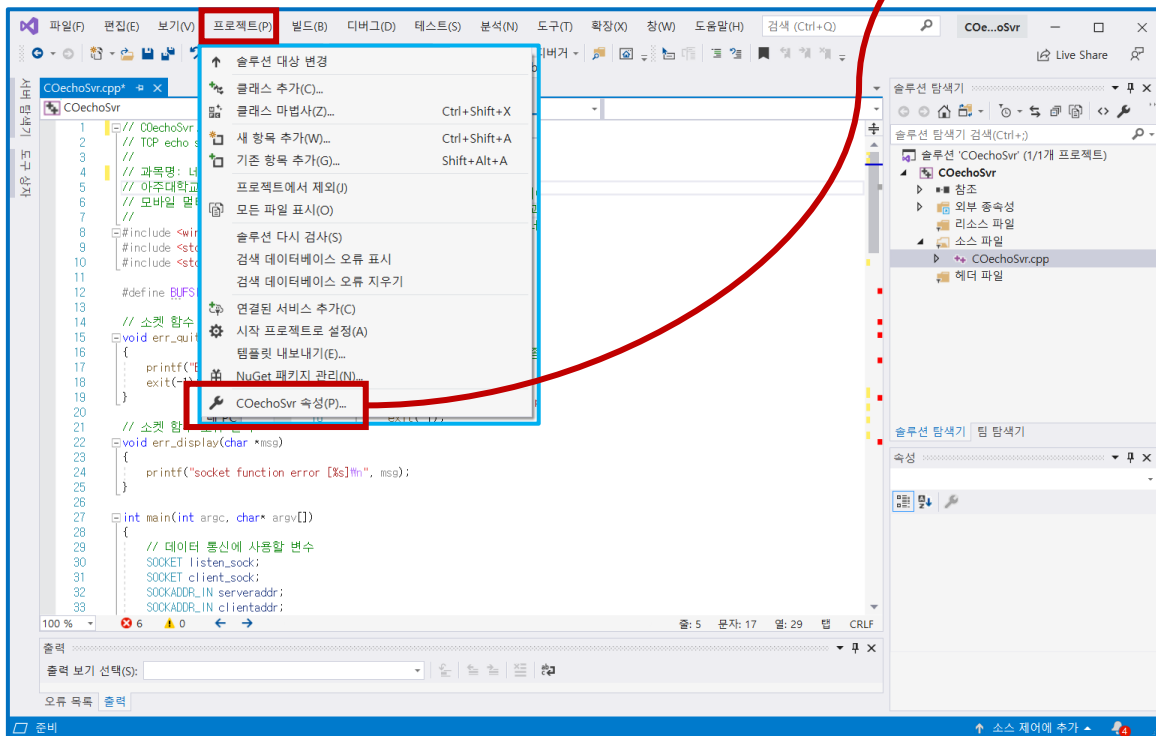
- ❖ Create a project for a Npcap programming
 - Microsoft Visual Studio



Npcap Programming with Visual Studio

❖ Library and Include Files

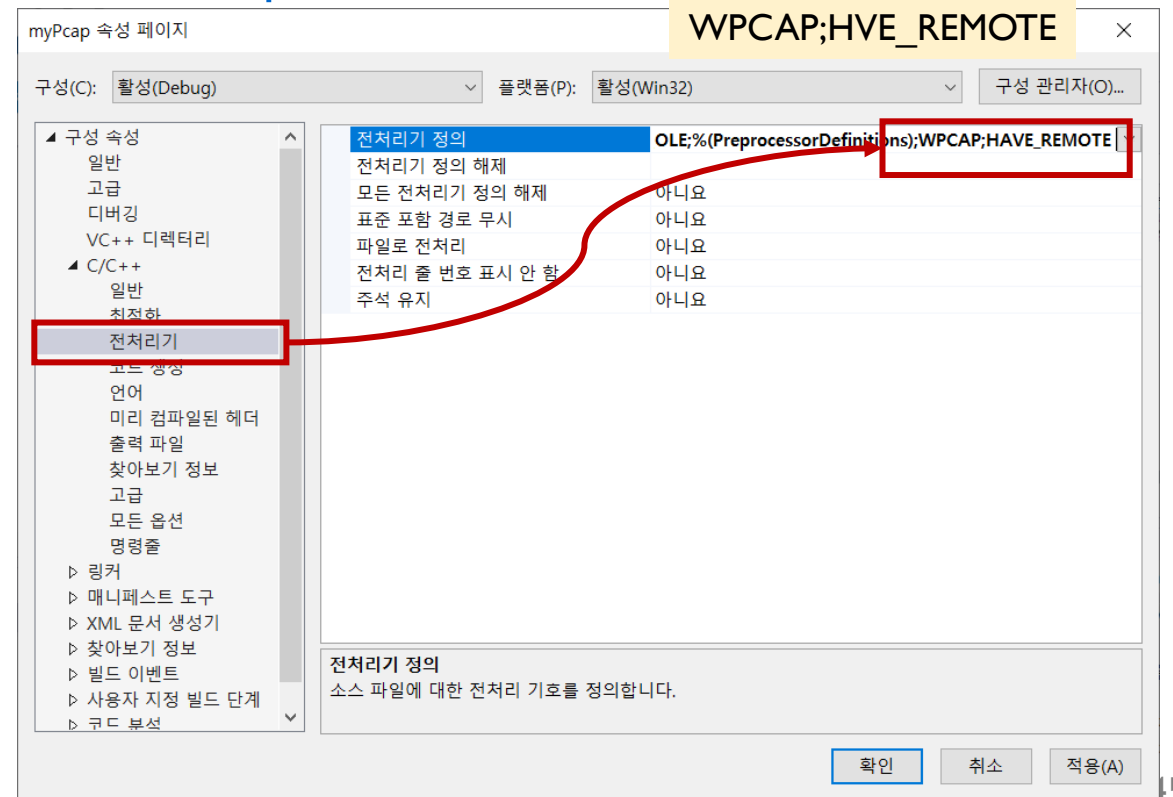
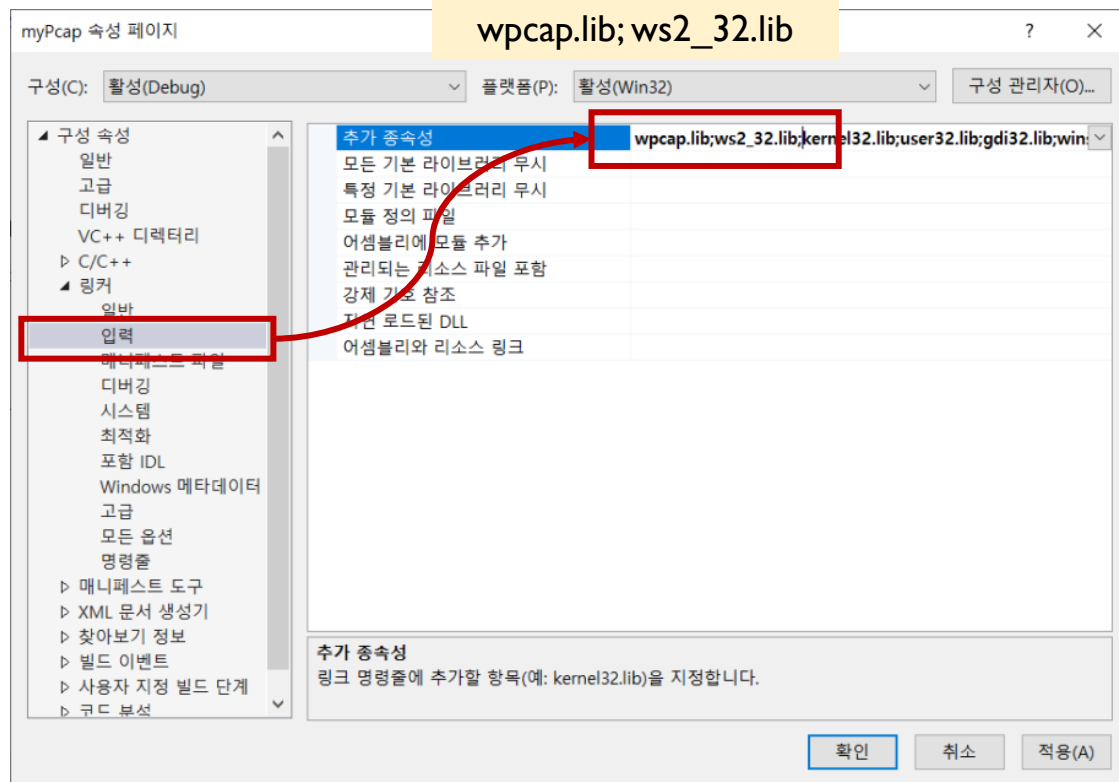
- add directories for include and library files



Npcap Programming with Visual Studio

❖ Set the linker options

- add wpcap.lib and ws2_32.lib
- define preprocessors
 - WPCAP for local and HAVE_REMOTE for remote captures





Npcap Functions

- ❖ Obtaining the list and advanced information about devices
- ❖ Capturing the packets from devices
- ❖ Capturing the packets without the callback
- ❖ Filtering the traffic
- ❖ Interpreting the packets
- ❖ Handling offline dump files
- ❖ Sending Packets
- ❖ Gathering Statistics on the network traffic

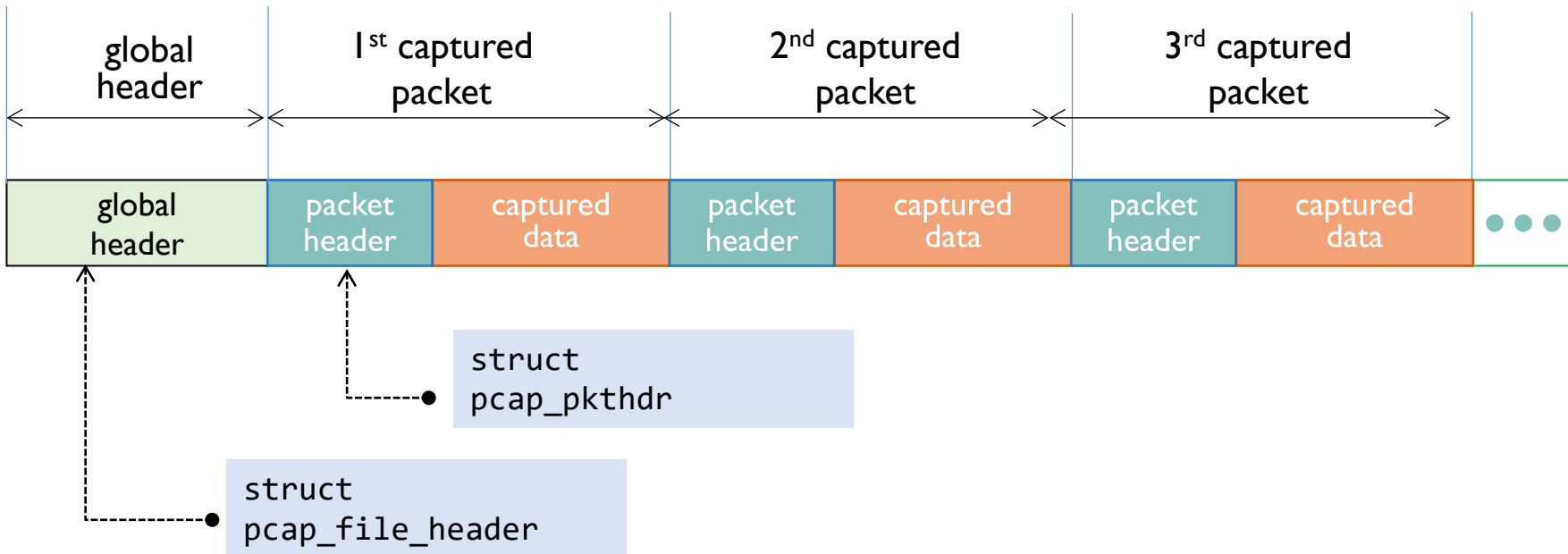


PCAP DATA STRUCTURE

Pcap Data Structure

❖ Data structure of captured file

- begins with global header, and
- repeats each captured packet with header and data



Pcap Data Structure

❖ Global header

- It contains metadata about the capture
- defined in <pcap.h>

```
struct pcap_file_header {  
    bpf_u_int32      magic;           // 0xA1B2C3D4 for a valid pcap file  
    u_short          version_major;  
    u_short          version_minor;  
    bpf_int32        thiszone;       // gmt to local correction - not used  
    bpf_u_int32      sigfigs;        // accuracy of timestamps - not used  
    bpf_u_int32      snaplen;        // max length saved portion of each pkt  
    bpf_u_int32      linktype;       // data link type (e.g. Ethernet, 802.11, ..)  
};
```

▪ macros

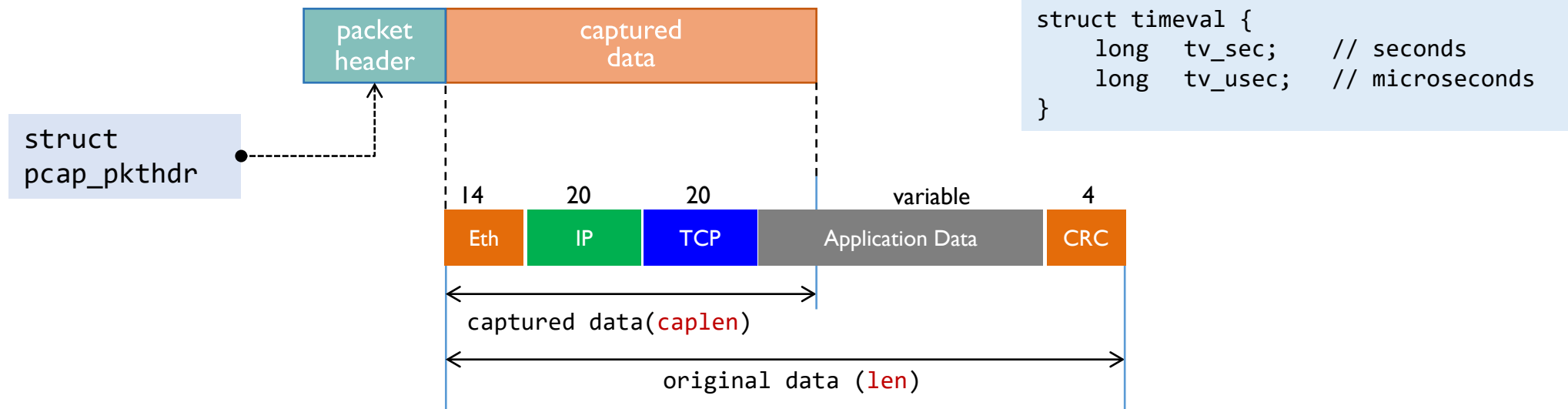
```
#define u_int32      bpf_u_int32  
#define int32        bpf_int32
```

Pcap Data Structure

❖ Packet header

- it contains the time and lengths information of each captured packet

```
struct pcap_pkthdr {  
    struct timeval    ts;        // time stamp (seconds and microseconds)  
    bpf_u_int32       caplen;    // length of captured packet (bytes)  
    bpf_u_int32       len;       // total length of the original packet (bytes)  
};
```



Wireshark Packet Capture Options

❖ Packet Capture Option - Input

capture Options

capture interface의 input 옵션 설정

caplen 지정 (1~262144)

WiFi packet capture in monitor mode

모든 패킷들을 캡처하도록 함 (미 설정시, 자신의 송수신 패킷들만 캡처됨)

특정 패킷들만 캡처하도록 지정함

The image shows the 'Wireshark · Capture Interfaces' dialog box with the 'Input' tab selected. The dialog is annotated with green boxes and red arrows pointing to specific settings. The 'Interface' list includes '로컬 영역 연결* 9' (selected), '이더넷', 'Wi-Fi', '로컬 영역 연결* 10', '로컬 영역 연결* 2', '이더넷 2', '로컬 영역 연결* 1', 'Adapter for loopback traffic capture', '로컬 영역 연결* 8', and 'USBPcap1'. The 'Traffic' column shows waveforms for selected interfaces. The 'Link-layer Head' column shows 'Ethernet' for most interfaces and 'BSD loopback' for the loopback adapter. The 'Promiscuous' column has checkboxes, all of which are checked. The 'Snaplen (B)' column has a value of '262144' for the selected interface, which is circled in red. The 'Buffer (MB)' column has a value of '2' for the selected interface. The 'Monitor Mode' column has a checkbox that is unchecked and circled in red. The 'Capture Filter' column has a filter 'tcp port http' entered in the text box below the table. The 'Enable promiscuous mode on all interfaces' checkbox is checked and circled in red. The 'Start' button is highlighted in blue.

Interface	Traffic	Link-layer Head	Promiscuous	Snaplen (B)	Buffer (MB)	Monitor Mode	Capture Filter
로컬 영역 연결* 9		Ethernet	☒	262144	2	☐	
> 이더넷		Ethernet	☒	default	2	☐	
> Wi-Fi		Ethernet	☒	default	2	☐	
로컬 영역 연결* 10		Ethernet	☒	default	2	☐	
로컬 영역 연결* 2		Ethernet	☒	default	2	☐	
> 이더넷 2		Ethernet	☒	default	2	☐	
로컬 영역 연결* 1		Ethernet	☒	default	2	☐	
Adapter for loopback traffic capture		BSD loopback	☒	default	2	☐	
로컬 영역 연결* 8		Ethernet	☒	default	2	☐	
USBPcap1		USBPcap	☐	—	—	☐	

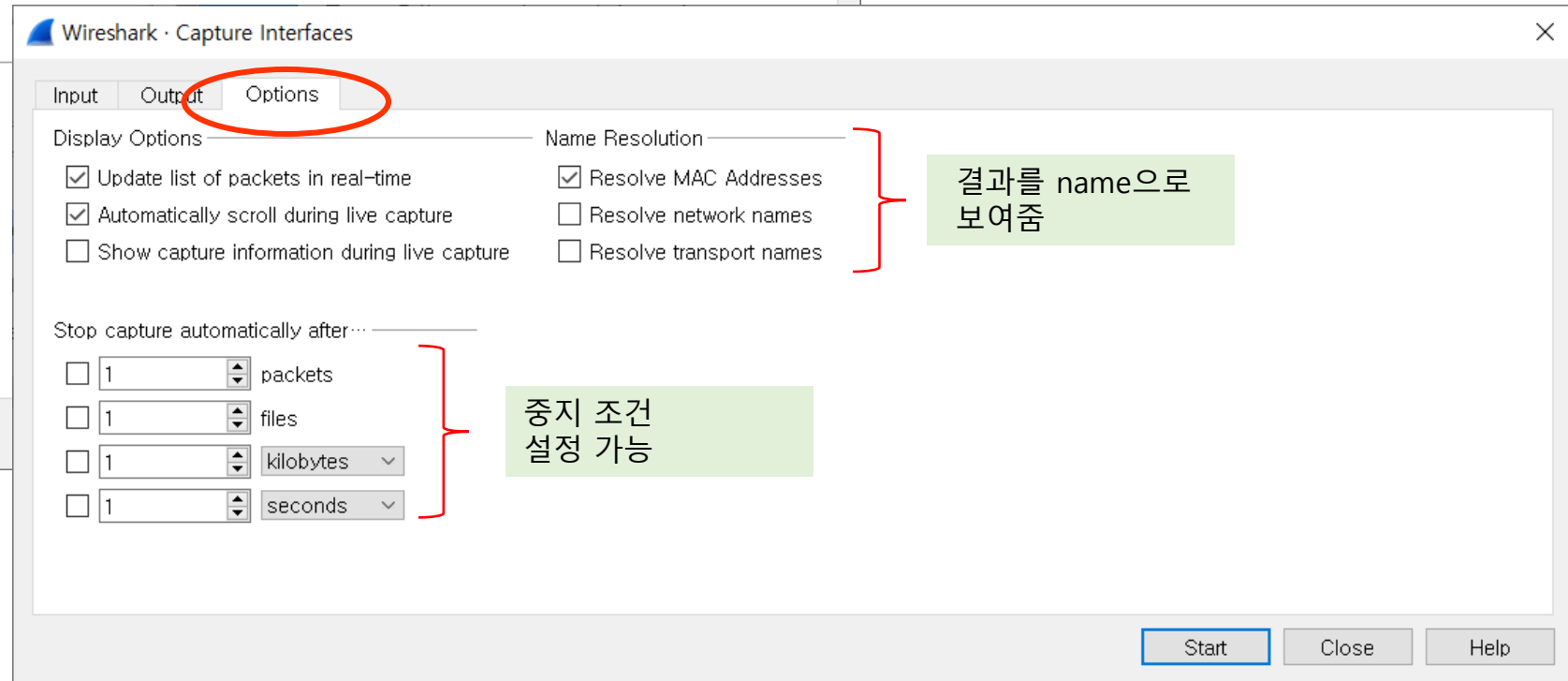
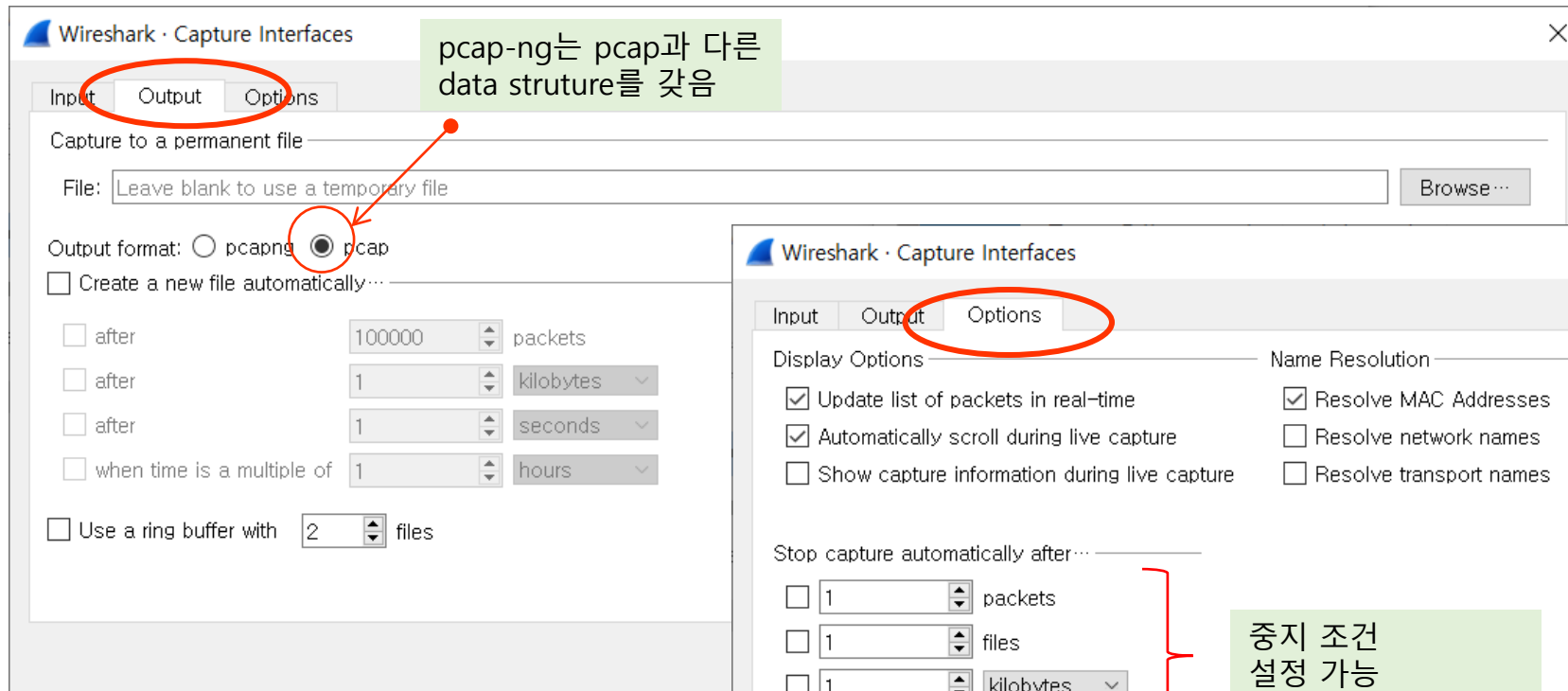
Enable promiscuous mode on all interfaces

Capture filter for selected interfaces: tcp port http

Start Close Help

Wireshark Packet Capture Options

❖ Packet Capture Option – Output and Options





PCAP PROGRAMMING EXAMPLE – CAPTURED FILE ANALYSIS



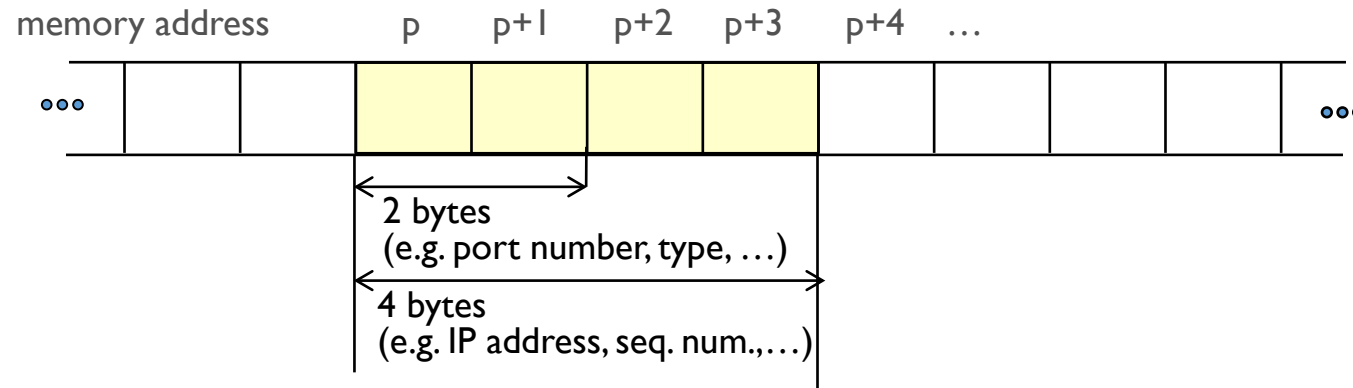
Captured File Analysis

❖ Procedure

- Step 1 : run WireShark
 - set appropriate parameters
- Step 2 : capture packets and store them to a file
 - run some networking applications such as ftp, ping, http, ...
- Step 3 : analyze capture file using capture file analysis program
 - open the capture file
 - read each packet according to pcap data structure
 - analyze each followings:
 - Ethernet frame header
 - IP datagram header
 - TCP/UDP header
 - Application protocol header or data
 - analyze more
 - connection information
 - application usage (time, number of exchanged packets, ...)

Tips on writing programs

❖ Reading a field with 2 or 4 bytes from a given memory address



- macro to read a value with 2 bytes

```
#define ntohs16(p)    ( (unsigned short)*((unsigned char *) (p)+0)<<8 | \
                        (unsigned short)*((unsigned char *) (p)+1)<<0 )
```

- macro to read a value with 4 bytes

```
#define ntohs32(p)    ( (unsigned short)*((unsigned char *) (p)+0)<<24 | \
                        (unsigned short)*((unsigned char *) (p)+1)<<16 | \
                        (unsigned short)*((unsigned char *) (p)+2)<<8  | \
                        (unsigned short)*((unsigned char *) (p)+3)<<0  )
```


Captured File Analysis

❖ Example code to get information from capture file

```
#include <pcap.h>

main( )
{
    struct pcap_file_hdr    fhdr;
    struct pcap_pkt_hdr    chdr;
    unsigned char           buffer[2000];
    FILE                    *trace;

    trace = fopen ("my_capture_file.cap","rb") ;

    fread ((char *)&fhdr, sizeof(fhdr), 1, trace);

    while ( fread ((char *)&chdr, sizeof(chdr), 1, trace) != 0 )
    {
        fread (buffer, sizeof(unsigned char), chdr.caplen, trace);
        do_traffic_analysis (buffer);
    }
    fclose(trace);
}
```

It should be greater than caplen

// file header in captured file
// packet header

// open capture file

// read the global file header

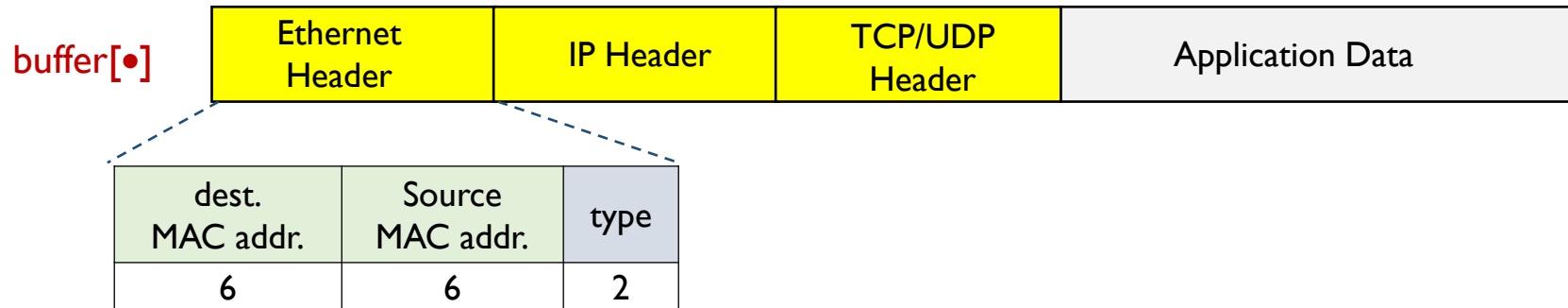
// read packet header

// read packet data

// main routing for capture packet analysis

Tips on writing programs

❖ Reading a field from a whole header



```
/* MAC address */
typedef struct mac_address {
    u_char byte1;
    u_char byte2;
    u_char byte3;
    u_char byte4;
    u_char byte5;
    u_char byte6;
} mac_address;
```

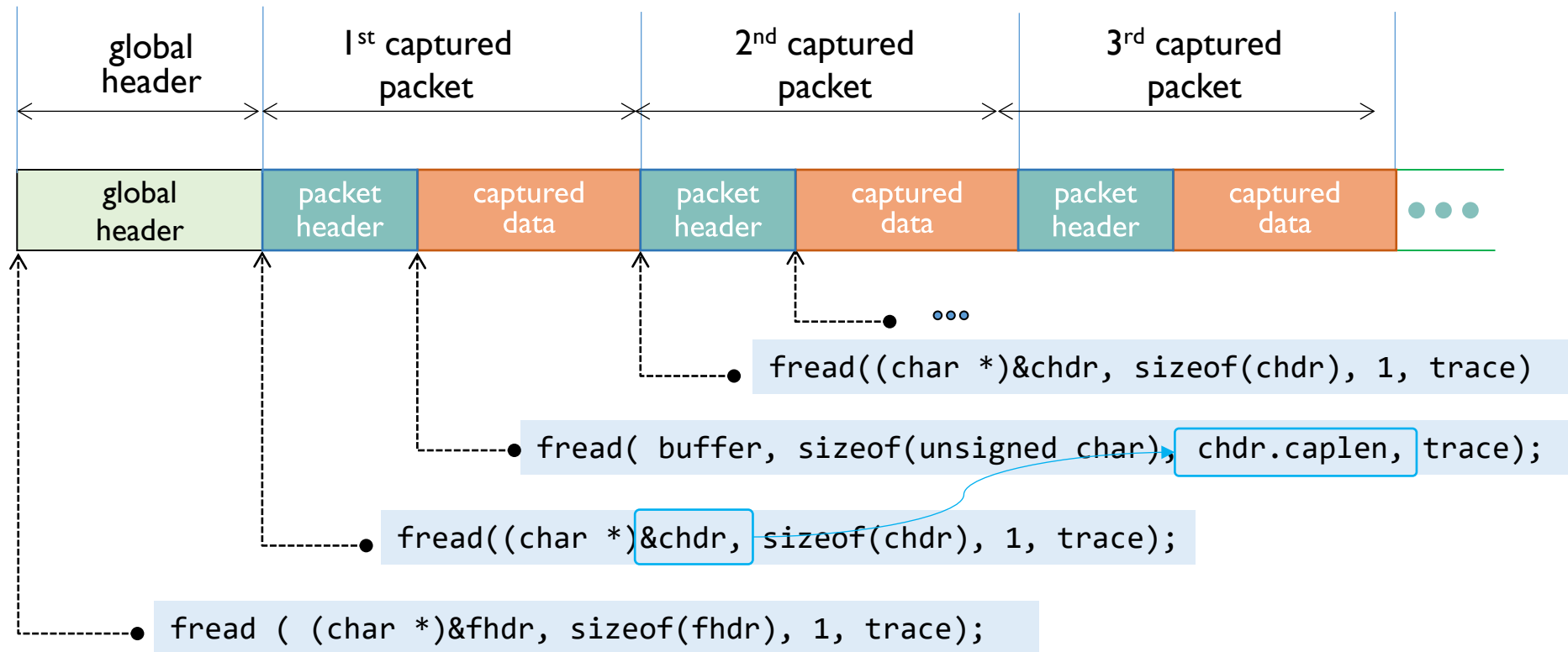
```
/* ethernet header */
typedef struct eth_header{
    mac_address    smac;
    mac_address    dmac;
    u_short        type;
} eth_header;

eth_header *eh;
eh = (eth_header *)(buffer);

if ( ntohs (eh->type) == 0x0800)
    printf("upper layer is IP\n");
```

Captured File Analysis

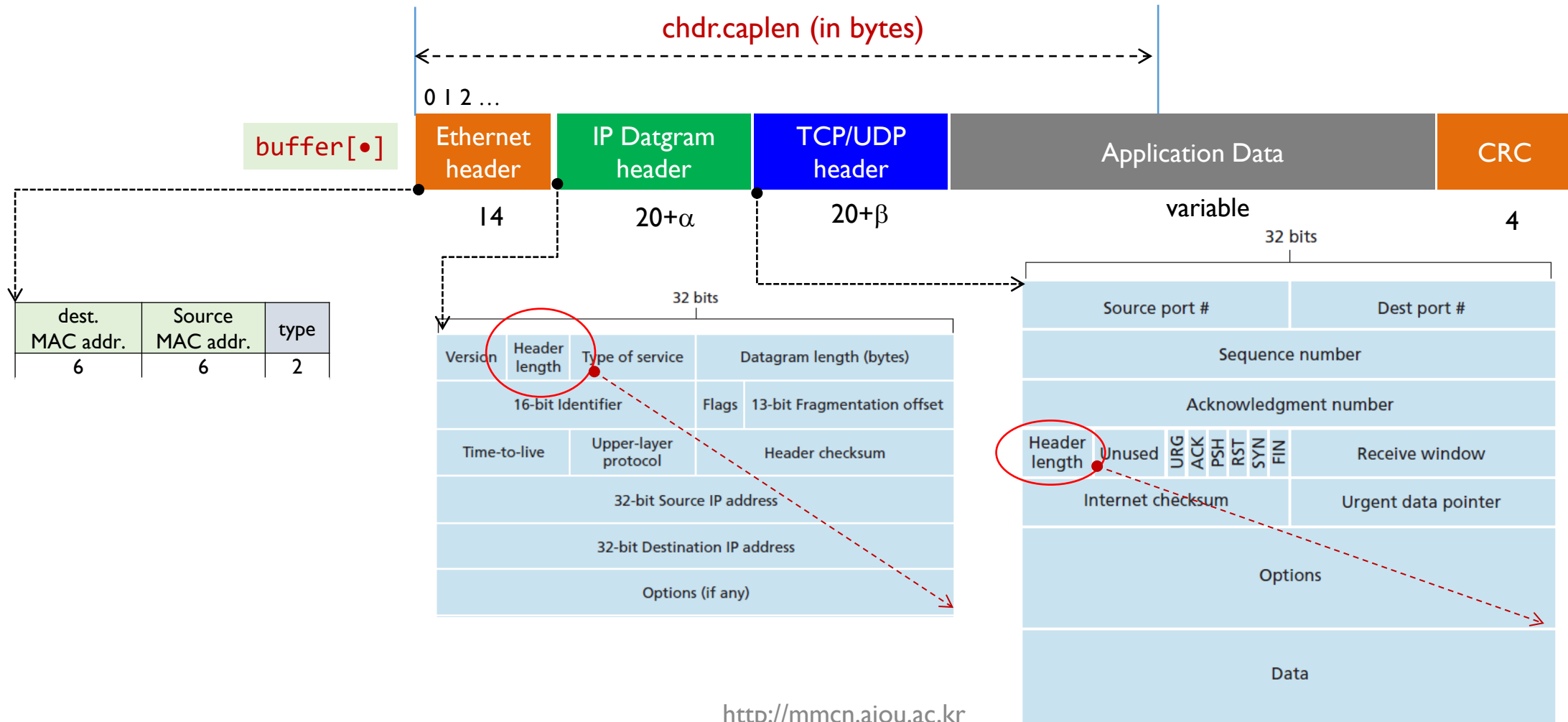
❖ Example – data structure



```
struct pcap_file_hdr  fhdr;  
struct pcap_pkt_hdr  chdr;
```

Captured File Analysis

❖ Example – buffer (packet data)



Example – CaptureFileAnalysis.c (1/5)

```
#include <pcap.h>
#include <stdio.h>
#include <stdlib.h>

#define LINE_LEN 16

// literals related to distinguishing protocols
#define ETHertype_IP      0x0800
#define ETH_II_HSIZE      14      // frame size of ethernet v2
#define ETH_802_HSIZE     22      // frame size of IEEE 802.3 ethernet
#define IP_PROTO_IP       0       // IP
#define IP_PROTO_TCP      6       // TCP
#define IP_PROTO_UDP      17      // UDP
#define RTPHDR_LEN        12      // Length of basic RTP header
#define CSRCID_LEN        4       // CSRC ID length
#define EXTHDR_LEN        4       // Extension header length

unsigned long    net_ip_count;
unsigned long    net_etc_count;
unsigned long    trans_tcp_count;
unsigned long    trans_udp_count;
unsigned long    trans_etc_count;
```

Example – CaptureFileAnalysis.c (2/5)

```
void main(int argc, char **argv)
{
    struct pcap_file_header    fhdr;
    struct pcap_pkthdr         chdr;
    unsigned char              buffer[262144];
    int                        i=0;

    FILE* fin = fopen( "ccc.pcap", "rb");

    fread ( (char *)&fhdr,  sizeof(fhdr), 1, fin);

    while ( fread( (char *)&chdr,  sizeof(chdr), 1, fin) != 0 ) { // Packet header
        fread(buffer, sizeof(unsigned char), chdr.caplen, fin); // Packet data
        do_traffic_analysis (buffer);
        i++;
    }
    fclose(fin);

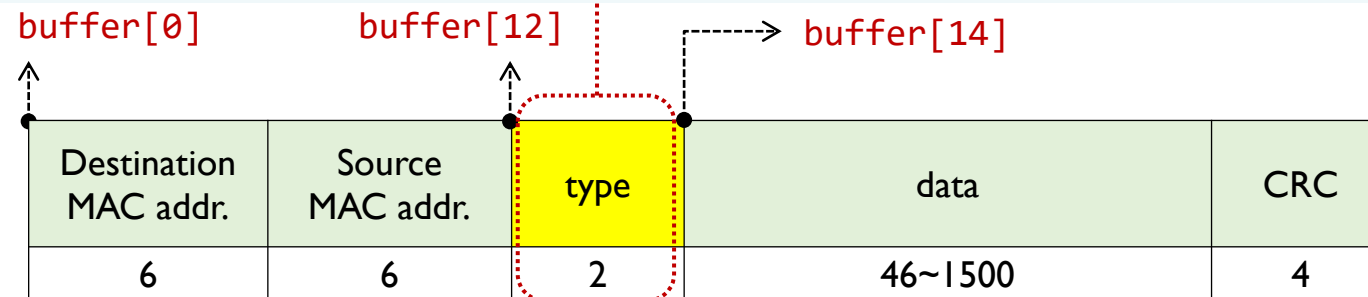
    printf("#total number of packets : %d\n", i);
    printf("IP packets: %d\n", net_ip_count);
    printf("non-IP packets: %d\n", net_etc_count);
    printf("TCP packets: %d\n", trans_tcp_count);
}
```

Example – CaptureFileAnalysis.c (3/5)

```
void do_traffic_analysis (unsigned char *buffer)
{
    unsigned short type;

    // ethernet type check
    type = ntohs(&buffer[12]);

    if ( type == ETHERTYPE_IP )
    {
        net_ip_count++;
        do_ip_traffic_analysis (buffer) ;
    }
    else
        net_etc_count++;
}
```

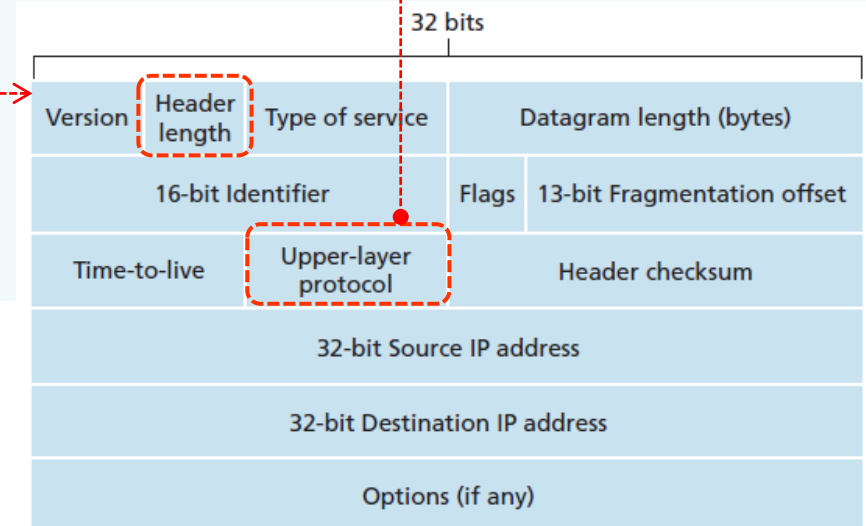
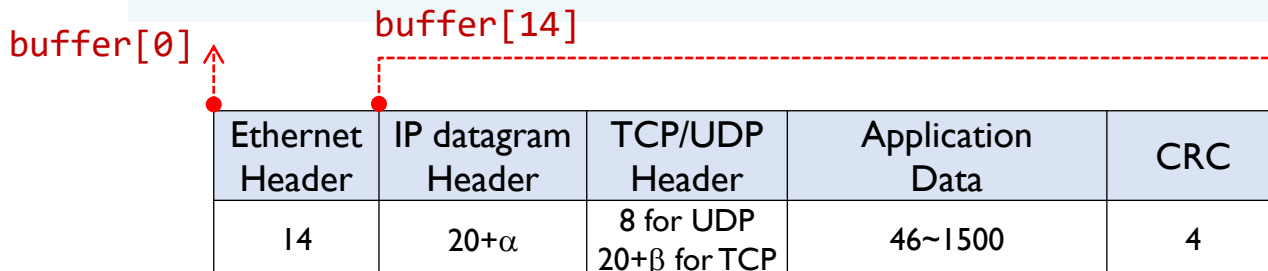


Example – CaptureFileAnalysis.c (4/5)

```
void do_ip_traffic_analysis (unsigned char *buffer)
{
    unsigned char    ip_ver, ip_hdr_len, ip_proto;
    int              ip_offset=14;

    ip_ver           = buffer[ip_offset]>>4;           // IP version
    ip_hdr_len       = buffer[ip_offset] & 0x0f ;       // IP header length
    ip_proto         = buffer[ip_offset + 9];           // protocol above IP

    if ( ip_proto == IP_PROTO_UDP )
        trans_udp_count++;
    else if ( ip_proto == IP_PROTO_TCP )
        trans_tcp_count++;
    else
        trans_etc_count++;
}
```

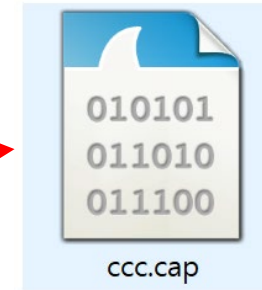
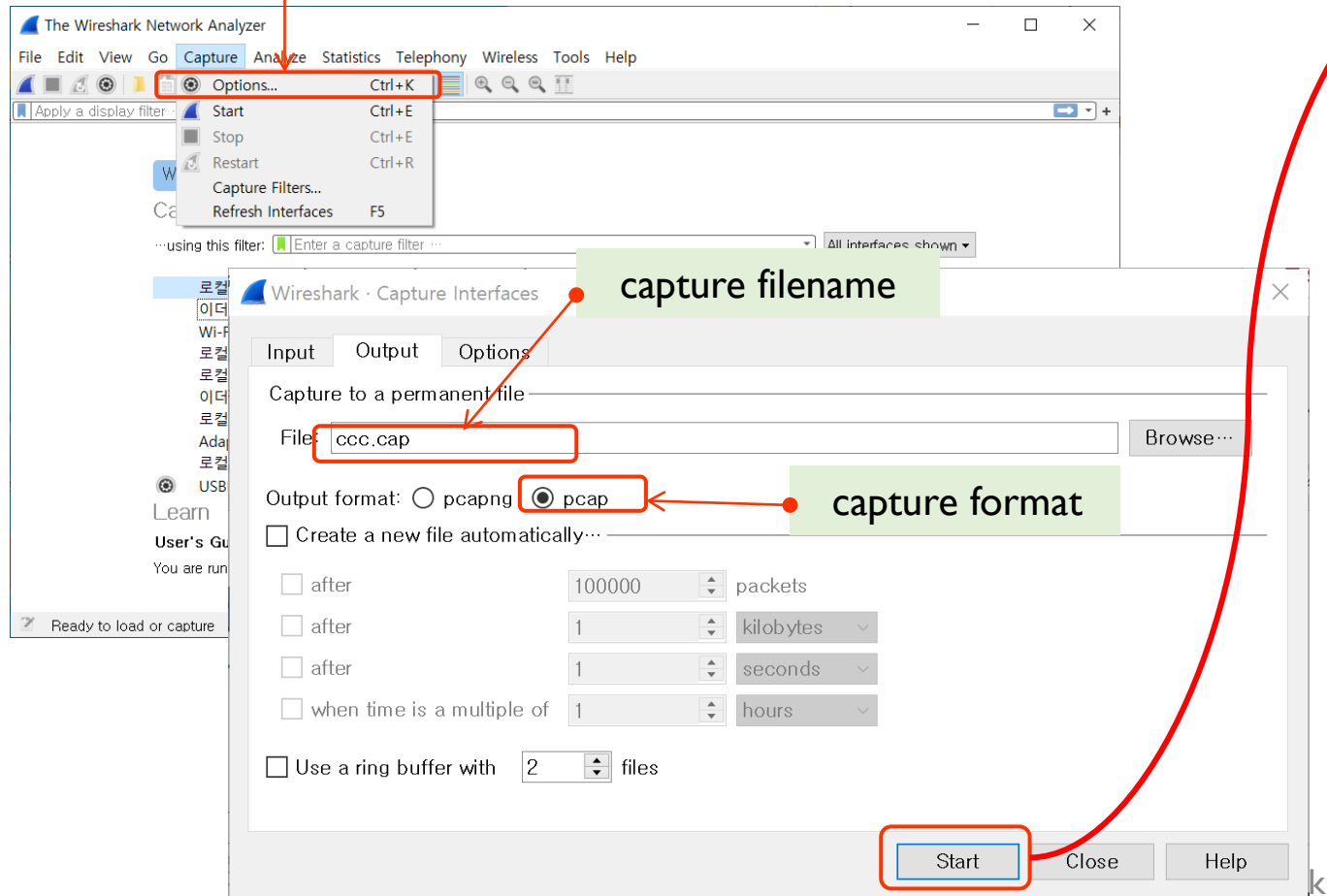


Example – CaptureFileAnalysis.c (5/5)



run

capture option



```
C:\> 명령 프롬프트

c:\W>mypcap
#total number of packets : 3000
IP packets: 2970
non-IP packets: 30
TCP packets: 2245

c:\W>
```



PCAP PROGRAMMING EXAMPLE – CAPTURED FILE ANALYSIS2

Simple Extension of CaptureFileAnalysis.c

❖ Use of packet header structures

```
/* ethernet header */
struct ethernet_header {
    uint8_t dest_mac[6];    // 목적지 MAC 주소
    uint8_t src_mac[6];     // 소스 MAC 주소
    uint16_t eth_type;      // Ethernet 타입
};
```

```
// IPv4 Header
struct ip_header {
    uint8_t  ihl : 4;        // IP 헤더 길이
    uint8_t  version : 4;    // IP 버전
    uint8_t  tos;            // 서비스 타입
    uint16_t tot_len;        // 전체 길이
    uint16_t id;            // 식별자
    uint16_t frag_off;      // 플래그 + 오프셋
    uint8_t  ttl;           // TTL
    uint8_t  protocol;      // 프로토콜 (TCP, UDP 등)
    uint16_t check;         // 체크섬
    uint32_t saddr;         // 소스 IP 주소
    uint32_t daddr;         // 목적지 IP 주소
};
```

Simple Extension of CaptureFileAnalysis.c

```
...
fread(&file_header, sizeof(struct pcap_file_header), 1, file);

while (fread(&packet_header, sizeof(struct pcap_pkthdr), 1, file) == 1) {

    fread(packet_data, packet_header.caplen, 1, file); // 패킷 데이터 읽기

    struct ethernet_header *eth_hdr = (struct ethernet_header *)packet_data; // Ethernet Header

    if (ntohs(eth_hdr->eth_type) == 0x0800) {

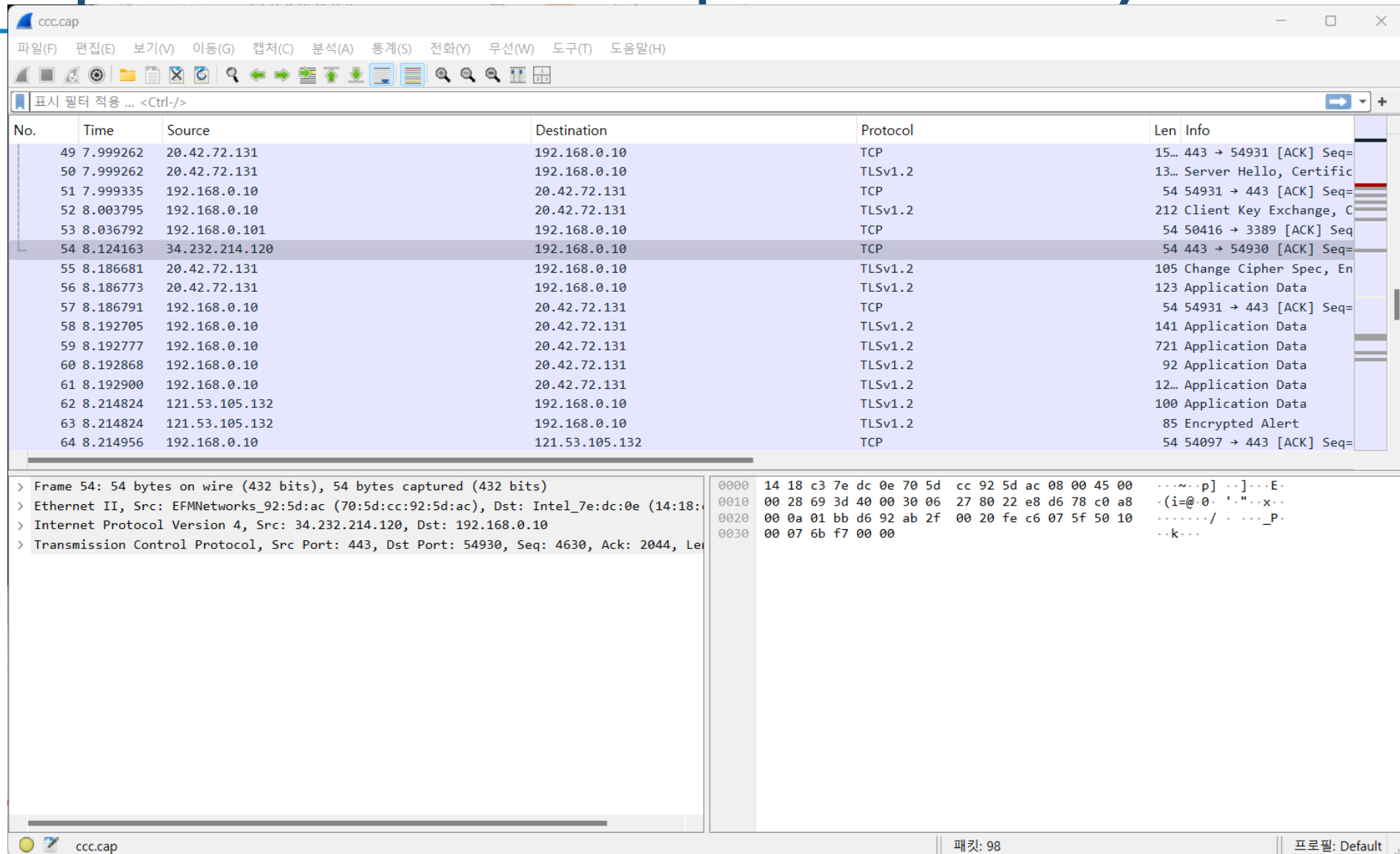
        struct ip_header *ip_hdr = (struct ip_header *)(packet_data + 14); // IP Header

        char src_ip[INET_ADDRSTRLEN], dest_ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &ip_hdr->saddr, src_ip, INET_ADDRSTRLEN);
        inet_ntop(AF_INET, &ip_hdr->daddr, dest_ip, INET_ADDRSTRLEN);

        const char *protocol;
        if ( ip_hdr->protocol == 6 )
            protocol = "TCP";

        printf("Captured length: %u, Actual length: %u\n", packet_header.caplen, packet_header.len);
        ...
    }
}
```

Simple Extension of CaptureFileAnalysis.c



The screenshot shows a network capture analysis tool interface. The top menu bar includes options like File (F), Edit (E), View (V), Go (G), Capture (C), Analysis (A), Statistics (S), Translate (Y), Wireless (W), Tools (T), and Help (H). Below the menu is a toolbar with various icons for file operations, capture, and analysis. A filter bar shows '표시 필터 적용 ... <Ctrl-/>'. The main window displays a list of captured packets with columns for No., Time, Source, Destination, Protocol, Len, and Info. Packet 54 is highlighted, showing a TCP connection from 34.232.214.120 to 192.168.0.10. Below the packet list, a detailed view of packet 54 is shown, including the frame structure, Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol header. The packet details show it is a TCP ACK packet with sequence number 443 and acknowledgment number 54930.

No.	Time	Source	Destination	Protocol	Len	Info
49	7.999262	20.42.72.131	192.168.0.10	TCP	15...	443 → 54931 [ACK] Seq=
50	7.999262	20.42.72.131	192.168.0.10	TLSv1.2	13...	Server Hello, Certific
51	7.999335	192.168.0.10	20.42.72.131	TCP	54	54931 → 443 [ACK] Seq=
52	8.003795	192.168.0.10	20.42.72.131	TLSv1.2	212	Client Key Exchange, C
53	8.036792	192.168.0.101	192.168.0.10	TCP	54	50416 → 3389 [ACK] Seq=
54	8.124163	34.232.214.120	192.168.0.10	TCP	54	443 → 54930 [ACK] Seq=
55	8.186681	20.42.72.131	192.168.0.10	TLSv1.2	105	Change Cipher Spec, En
56	8.186773	20.42.72.131	192.168.0.10	TLSv1.2	123	Application Data
57	8.186791	192.168.0.10	20.42.72.131	TCP	54	54931 → 443 [ACK] Seq=
58	8.192705	192.168.0.10	20.42.72.131	TLSv1.2	141	Application Data
59	8.192777	192.168.0.10	20.42.72.131	TLSv1.2	721	Application Data
60	8.192868	192.168.0.10	20.42.72.131	TLSv1.2	92	Application Data
61	8.192900	192.168.0.10	20.42.72.131	TLSv1.2	12...	Application Data
62	8.214824	121.53.105.132	192.168.0.10	TLSv1.2	100	Application Data
63	8.214824	121.53.105.132	192.168.0.10	TLSv1.2	85	Encrypted Alert
64	8.214956	192.168.0.10	121.53.105.132	TCP	54	54097 → 443 [ACK] Seq=

Frame 54: 54 bytes on wire (432 bits), 54 bytes captured (432 bits)
 Ethernet II, Src: EFMNetworks_92:5d:ac (70:5d:cc:92:5d:ac), Dst: Intel_7e:dc:0e (14:18:00:03:27:00)
 Internet Protocol Version 4, Src: 34.232.214.120, Dst: 192.168.0.10
 Transmission Control Protocol, Src Port: 443, Dst Port: 54930, Seq: 4630, Ack: 2044, Len: 0

0000 14 18 c3 7e dc 0e 70 5d cc 92 5d ac 08 00 45 00 ...~..p] ..]...E.
 0010 00 28 69 3d 40 00 30 06 27 80 22 e8 d6 78 c0 a8 ..(i=@.0. '..."..x..
 0020 00 0a 01 bb d6 92 ab 2f 00 20 fe c6 07 5f 50 10/_P..
 0030 00 07 6b f7 00 00 ...k...

Simple Extension of CaptureFileAnalysis.c

```
Microsoft Visual Studio 디버그 × + ▾  
[37] Source IP: 34.232.214.120, Destination IP: 192.168.0.10, Protocol: TCP  
[38] Source IP: 192.168.0.10, Destination IP: 34.232.214.120, Protocol: TCP  
[39] Source IP: 192.168.0.10, Destination IP: 34.232.214.120, Protocol: TCP  
[40] Source IP: 20.42.72.131, Destination IP: 192.168.0.10, Protocol: TCP  
[41] Source IP: 192.168.0.10, Destination IP: 20.42.72.131, Protocol: TCP  
[42] Source IP: 192.168.0.10, Destination IP: 20.42.72.131, Protocol: TCP  
[43] Source IP: 192.168.0.10, Destination IP: 151.101.88.157, Protocol: TCP  
[44] Source IP: 151.101.88.157, Destination IP: 192.168.0.10, Protocol: TCP  
[45] Source IP: 34.232.214.120, Destination IP: 192.168.0.10, Protocol: TCP  
[46] Source IP: 192.168.0.10, Destination IP: 34.232.214.120, Protocol: TCP  
[47] Source IP: 192.168.0.10, Destination IP: 192.168.0.101, Protocol: TCP  
[48] Source IP: 20.42.72.131, Destination IP: 192.168.0.10, Protocol: TCP  
[49] Source IP: 20.42.72.131, Destination IP: 192.168.0.10, Protocol: TCP  
[50] Source IP: 20.42.72.131, Destination IP: 192.168.0.10, Protocol: TCP  
[51] Source IP: 192.168.0.10, Destination IP: 20.42.72.131, Protocol: TCP  
[52] Source IP: 192.168.0.10, Destination IP: 20.42.72.131, Protocol: TCP  
[53] Source IP: 192.168.0.101, Destination IP: 192.168.0.10, Protocol: TCP  
[54] Source IP: 34.232.214.120, Destination IP: 192.168.0.10, Protocol: TCP  
[55] Source IP: 20.42.72.131, Destination IP: 192.168.0.10, Protocol: TCP  
[56] Source IP: 20.42.72.131, Destination IP: 192.168.0.10, Protocol: TCP  
[57] Source IP: 192.168.0.10, Destination IP: 20.42.72.131, Protocol: TCP  
[58] Source IP: 192.168.0.10, Destination IP: 20.42.72.131, Protocol: TCP  
[59] Source IP: 192.168.0.10, Destination IP: 20.42.72.131, Protocol: TCP  
[60] Source IP: 192.168.0.10, Destination IP: 20.42.72.131, Protocol: TCP  
[61] Source IP: 192.168.0.10, Destination IP: 20.42.72.131, Protocol: TCP  
[62] Source IP: 121.53.105.132, Destination IP: 192.168.0.10, Protocol: TCP  
[63] Source IP: 121.53.105.132, Destination IP: 192.168.0.10, Protocol: TCP  
[64] Source IP: 192.168.0.10, Destination IP: 121.53.105.132, Protocol: TCP  
[65] Source IP: 192.168.0.10, Destination IP: 121.53.105.132, Protocol: TCP  
[66] Source IP: 121.53.105.132, Destination IP: 192.168.0.10, Protocol: TCP
```

Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

